

StudyPilot Stage 1 Design Report

EECS3311 Course Project

Student Name: Qi Zhou

Student Number: 216439747

1 Project overview

Problem

University students collect lecture notes, slides and readings from many courses. Turning this material into active study is time-consuming and manual, and students often do not know which topics they are weak in until the exam.

Target Users

- Primary: university students managing several courses.
- Secondary: self-learners who study from their own documents.

Agent Description

StudyPilot allows a student to create courses, import study materials, and study with an assistant agent that can summarize the materials, answer questions with citations, generate flashcards and practice tests, grade answers, identify weak spots, and develop a personalized study plan - each available on its own screen. The student can also make a free-form request; the agent then figures out a sequence of tool calls, executes them, verifies each step, and recovers if any step breaks.

Why an AI Agent

- Requests can be open-ended and involve selecting from multiple capabilities (retrieval, quiz generation, grading, planning).
- The right steps depend on the student's data (weak topics, deadlines), so a fixed script is not enough.
- An LLM alone cannot reliably track the student's progress or grade multiple-choice questions, and ordinary code alone cannot understand free-form course materials or explain answers. StudyPilot combines the two.

AI/LLM Model

- StudyPilot uses OpenAI GPT-6.1 Sol (gpt-6.1-sol) through the OpenAI API.

How the AI Interacts with the Software

The user interface never calls the LLM directly. The services send prompts to the model through one LLM client class and turn its reply into the program's own data. For the study assistant, the LLM proposes which tools to call, and the program checks each step before answering.

Overall architecture

SwingGUI and CLI call AgentFacade. The Facade either delegates to a domain service or forwards a natural-language request to AgentController. The controller uses AgentPlanner, ToolManager and MemoryManager. Services read and save data through five small JSON repository classes. All LLM calls go through LLMClient and OpenAIAdapter.

2 Feature specifications

Deterministic means no LLM is used, AI-based means an LLM does the main work, and Hybrid means both are used.

F01 - Course and Material Import

Description: Create a course and import study materials (.txt, .md, .pdf). The system extracts text and stores chunks for retrieval. Every chunk keeps its provenance (file name and page number for PDF, or line range for text/Markdown) so the student can go back to the original.

User Interaction: GUI: Courses → New Course → Add Material (file chooser). CLI: studypilot course add, studypilot material import <file>.

Input: Course name; file path(s).

Output: Course with stored Material and Chunks (each with chunkId, sourceFile, page/lineRange); import summary (pages, chunks).

AI Involvement: Deterministic

Workflow: Validate file → extract text → clean → chunk with overlap, recording source location → persist for RetrievalStrategy to search.

Errors: Unsupported/corrupt/empty file → reject with message; duplicate file (same content hash) → warn and skip; very large file → size-limit message.

F02 - Material Summarization

Description: Summarize a material in a chosen style (brief, detailed, bullet points).

User Interaction: GUI: select material → Summarize → choose style. CLI: studypilot summarize <materialId> --style bullet.

Input: Material id; style.

Output: Summary text stored with the material.

AI Involvement: AI-based

Workflow: PromptTemplates.summaryPrompt makes the prompt for the chosen style with the chunks → LLMClient → SummaryService checks the reply and summary text → store and display.

Errors: LLM timeout/error → retry once, then show error; empty or malformed response → regenerate; material too long → summarize chunk by chunk and merge.

F03 - Concept Extraction

Description: Extract key concepts with short definitions from materials.

User Interaction: GUI: Material → Extract Concepts; results shown in a list/tree. CLI: studypilot concepts extract <materialId>.

Input: Material id.

Output: List of Concepts, each with a name, short definition, optional parent concept name, and source chunk ids. The GUI may show them as a tree using the parent name.

AI Involvement: Hybrid

Workflow: LLM returns structured JSON → ResponseParser validates schema → merge duplicates → save the List<Concept> in the course.

Errors: Invalid JSON → retry with stricter prompt; no concepts found → inform user; cyclic relation → drop offending link.

F04 - Question Answering with Verified Citations

Description: Answer natural-language questions using only the student's materials. Citations are checked by the program, not just appended.

User Interaction: GUI: Ask panel in a course. CLI: studypilot ask "<question>" --course <id>.

Input: Question; course scope.

Output: Answer plus a list of citations that point back to the original.

AI Involvement: Hybrid

Workflow: Retrieve top-k chunks via RetrievalStrategy → build prompt containing chunks with their chunkIds and the constraint "answer only from context; return JSON {answer, citedChunkIds}" → LLM → CitationValidator checks that every cited id belongs to the retrieved set → invalid ids are removed → map valid ids to file/page for display.

Errors: No relevant chunks → say the materials do not cover it; retrieval failure → inform user; LLM cites unknown ids → remove them, and if no valid citation remains, show the answer as "uncited" or retry once; ambiguous question → ask a clarifying question.

F05 - Flashcards and Review

Description: Generate flashcards from a material, then review them: the student flips a card and marks it "remembered" or "not yet". The system records how well each card is known.

User Interaction: GUI: Flashcards → Generate. Review session shows cards to review; student flips and clicks Remembered / Not yet. CLI: studypilot flashcards generate <materialId> --count 20, studypilot review <courseId>.

Input: Generate: material, number of cards. Review: card id, remembered.

Output: List of Flashcards; after a review, the card's reviewCount, rememberedCount and mastery.

AI Involvement: Hybrid

Workflow: Generate: build prompt → LLM returns cards → parse and validate → save to the course. Review: pick cards to review → student marks remembered or not → update the card's counts → save → ProgressTracker.recordReview publishes a StudyEvent.

Errors: Fewer cards than requested → report actual count; duplicates removed; malformed output → retry; no cards to review → show "nothing to review"; unknown card id → rejected.

F06 - Quiz Generation

Description: Generate a quiz with mixed question types (multiple choice, true/false, short answer) for a topic. The difficulty comes from F07. A generated quiz is immutable; each time the student takes it, a new QuizAttempt is created.

User Interaction: GUI: Quiz → New Quiz (topic, count, question types, difficulty shown). CLI: studypilot quiz create --topic <t> --count 10.

Input: Topic/concepts; number; question types; difficulty.

Output: Quiz containing Question objects with answers/rubrics.

AI Involvement: Hybrid

Workflow: Get difficulty from F07 → retrieve context → LLM drafts questions → the matching QuestionCreator builds and validates each typed Question → create Quiz.

Errors: Invalid question (no correct option, duplicate options) → discard and regenerate; not enough material → fewer questions with notice.

F07 - Adaptive Question Difficulty

Description: Choose the difficulty of a new quiz from the student's overall accuracy on a topic so far, using a simple rule in QuizService.chooseDifficulty. The student can override the result.

User Interaction: GUI: the New Quiz screen shows the suggested difficulty with a short reason (e.g. "accuracy 85% over 12 questions") and a selector to override it. CLI: studypilot quiz difficulty --topic <t>.

Input: Topic; graded results for that topic in the course, from ProgressTracker.

Output: Difficulty level (Easy / Medium / Hard) with the accuracy and number of results it was based on.

AI Involvement: Deterministic

Workflow: ProgressTracker.getAccuracy(courseId, topic) → QuizService.chooseDifficulty(accuracy, gradedCount) → return level and reason → used by F06.

Errors: No history or too few graded results → default Medium and say so; invalid override value → ignored with message.

F08 - Quiz Attempts, Grading and Explanation

Description: Grade a student's attempt: multiple-choice and true/false deterministically; short answers by LLM against a rubric. Explain wrong answers.

User Interaction: GUI: Take Quiz → Submit → results with explanations. CLI: studypilot quiz take <quizId>, studypilot quiz submit <attemptId> --answers <file>.

Input: Quiz; student answers.

Output: Score, per-question result, explanation.

AI Involvement: Hybrid

Workflow: Start a new QuizAttempt → the student answers and submits → QuizGrader grades each answer (ExactMatchGrading for multiple-choice and true/false, LLMRubricGrading for short answers) → save the attempt → ProgressTracker records the graded results → show the score and explanations. A short answer that cannot be graded stays PENDING and can be re-graded or self-graded later.

Errors: Unanswered question → scored 0 and flagged; LLM grader failure → that question becomes PENDING; resubmitting an already graded attempt → returns the stored result.

F09 - Weak Topic Identification

Description: Detect weak topics from graded quiz results and explain why. Flashcard mastery is recorded separately (F05) and is not part of this ranking.

User Interaction: GUI: Progress → Weak Topics panel. CLI: studypilot weak-topics --course <id>.

Input: Graded QuestionResults per concept.

Output: Ranked list of weak topics (lowest accuracy first) with accuracy and number of results, plus a short AI explanation.

AI Involvement: Hybrid

Workflow: Compute accuracy per concept = correct ÷ graded results → rank from lowest → LLM writes explanation and suggestions → display.

Errors: A concept with fewer than N graded results (configurable, e.g. 5) is reported as "insufficient data", not ranked; LLM failure → show the ranking without explanation.

F10 - Personalized Study Plan Generation

Description: Build a day-by-day revision schedule (StudyPlan of StudyTasks) from deadlines, available time, weak topics and materials. This is a student schedule, not the agent's tool-execution plan.

User Interaction: GUI: Plan → New Plan (exam date, hours/day), then tick completed tasks. CLI: studypilot plan --exam <date> --hours 2, studypilot plan complete <planId> <taskId>.

Input: Exam/deadline dates; daily time; course.

Output: StudyPlan: dated tasks (read, review cards, take quiz) that the student can mark done.

AI Involvement: Hybrid

Workflow: StudyPlanGenerator reads weak topics (F09) and materials → LLM proposes tasks → StudyPlanGenerator.validatePlan checks (dates in range, positive task durations, daily hours not exceeded) → repair or regenerate if violated → save.

Errors: Exam date in the past → rejected; infeasible time budget → warn and propose reduced scope; no learning history → no "weakest topic" is claimed (schedule a diagnostic quiz first or ask the student to pick topics); plan violates constraints → repair or regenerate (bounded retries).

F11 - Progress Tracking and Session History

Description: Track study activity (reviews, graded quizzes, plan completion) and keep a history of study sessions.

User Interaction: GUI: Dashboard with progress per course, history list. CLI: studypilot progress, studypilot history.

Input: StudyEvents from other features.

Output: Progress counts (reviews done, quizzes taken, tasks done) and study history.

AI Involvement: Deterministic

Workflow: Features publish StudyEvent → ProgressTracker updates stats → observers (DashboardView, HistoryRecorder) refresh.

Errors: Unreadable progress or history file → keep a .bak copy, start with empty data and warn the student.

F12 - Natural Language Study Assistant

Description: Core agent: understands a free-form request, builds an ExecutionPlan, runs tools, checks each result and decides what to do next, uses conversation memory, and answers.

User Interaction: GUI: Assistant chat panel. CLI: studypilot agent "<request>" [--session <id>] [--json].

Input: Natural-language request; session id.

Output: Final answer plus execution trace (plan, tool calls, results, decisions, errors).

AI Involvement: AI-based

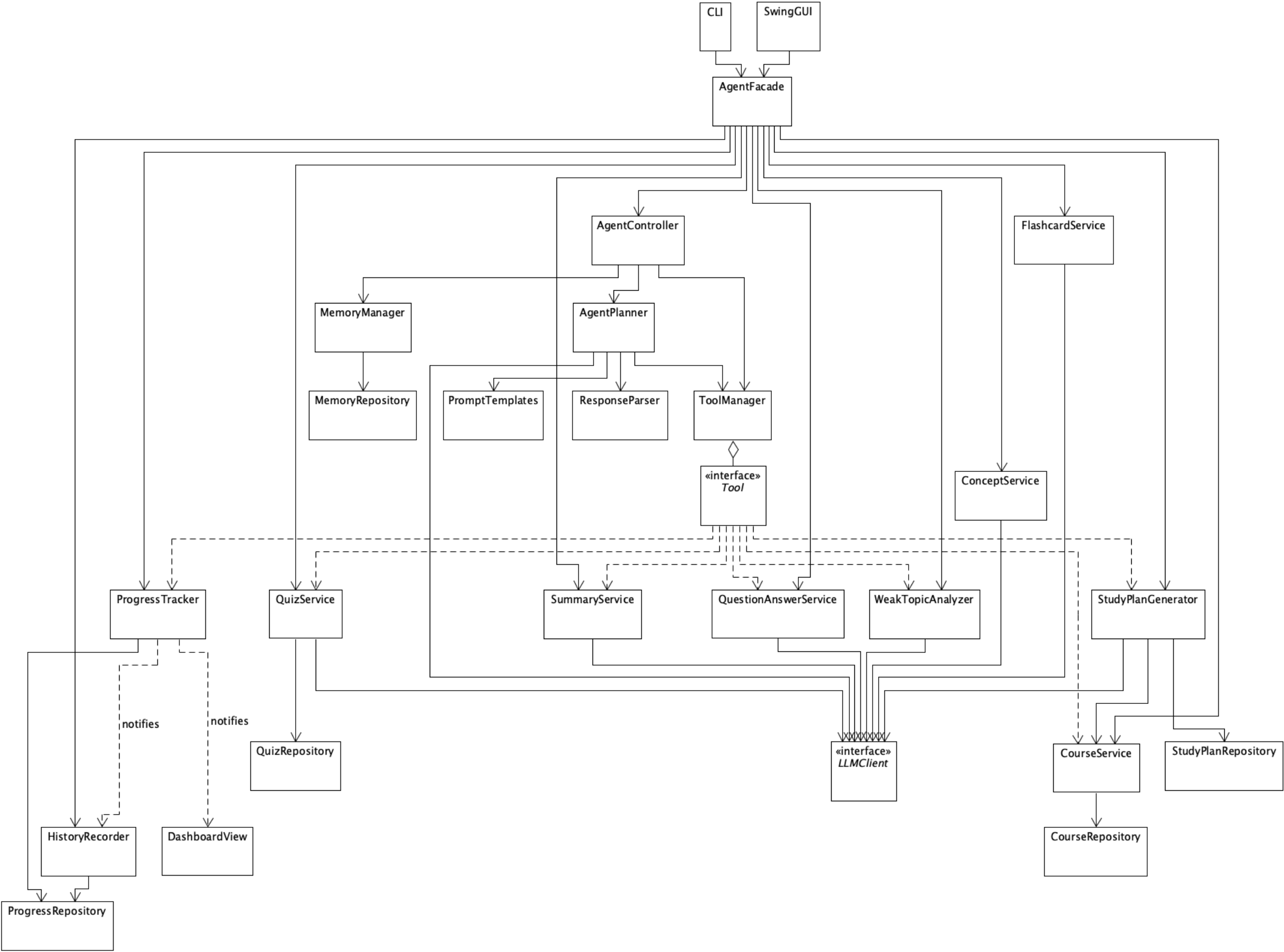
Workflow: AgentController loads the session history from MemoryManager → AgentPlanner asks the LLM for an ExecutionPlan, and the program checks it (tool exists, valid arguments, at most 8 steps) → ToolManager runs each step and returns a ToolResult (OK, FAILED or INSUFFICIENT_DATA) → AgentController.decide chooses CONTINUE, RETRY (max 2 per step), REPLAN (max 2 per request), CLARIFY or ABORT → the final answer is made from the tool results and returned with a trace.

Errors: Unknown tool or invalid arguments → the step is not run and the agent replans; tool failure → retry, then replan; not enough data → ask the student a question; limits reached → stop and report the partial result honestly.

3 UML class model

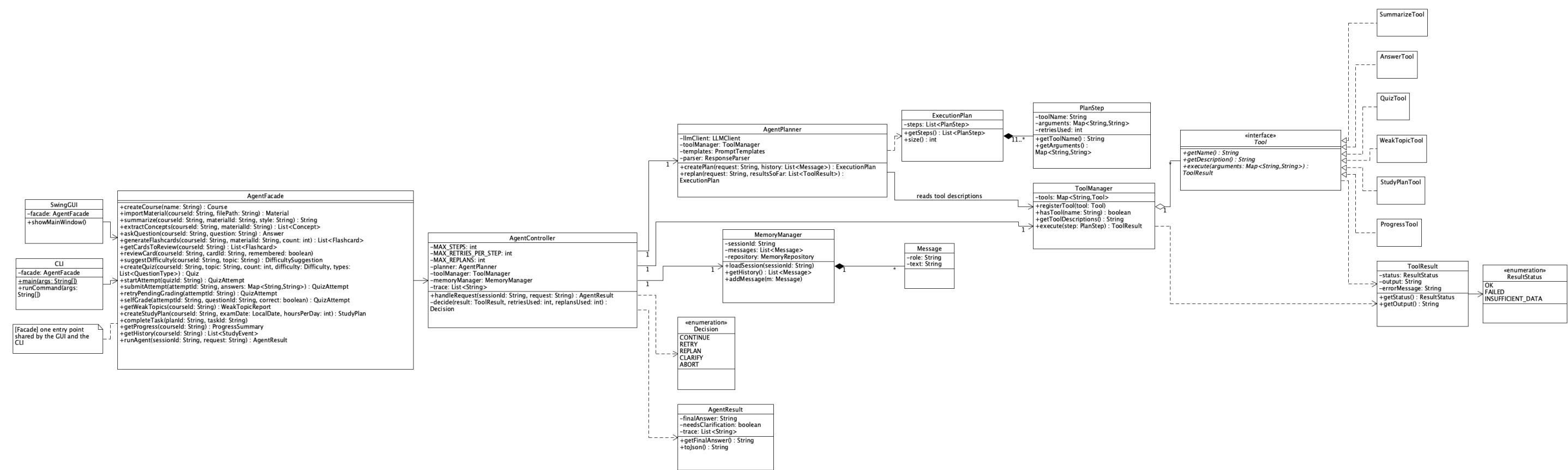
The class model is shown in 8 views. View 0 shows the main relationships and Views 1–7 show the details. Solid lines are associations, hollow triangles are inheritance or realization, diamonds are whole/part, and dotted arrows are dependencies. Association ends show multiplicity. PNG images and editable UMLet files are included.

View 0: Architecture overview (class names only)



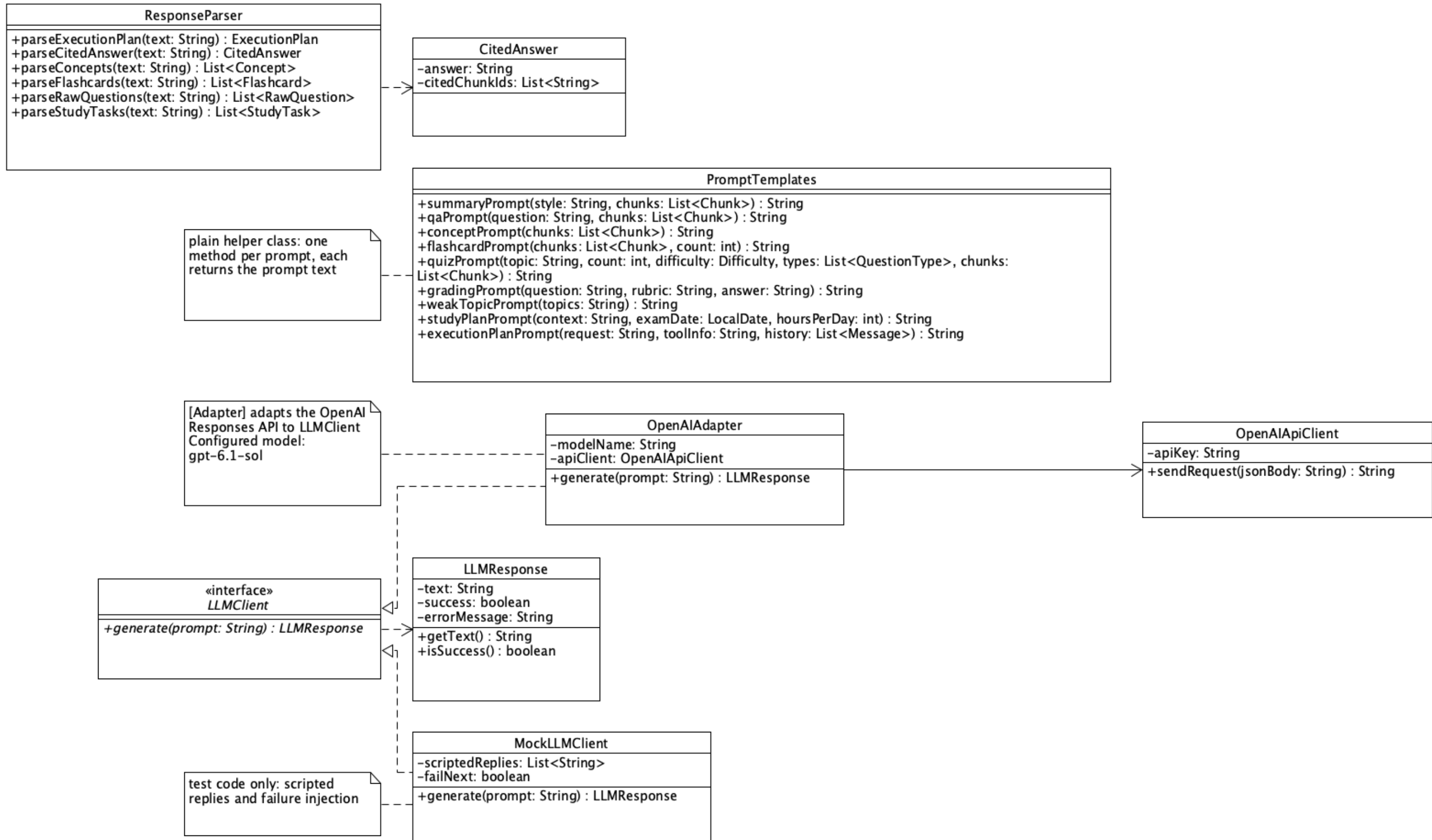
class-view-0 Complete diagram. Use zoom to read detailed labels.

View 1: Presentation and agent core (F12)



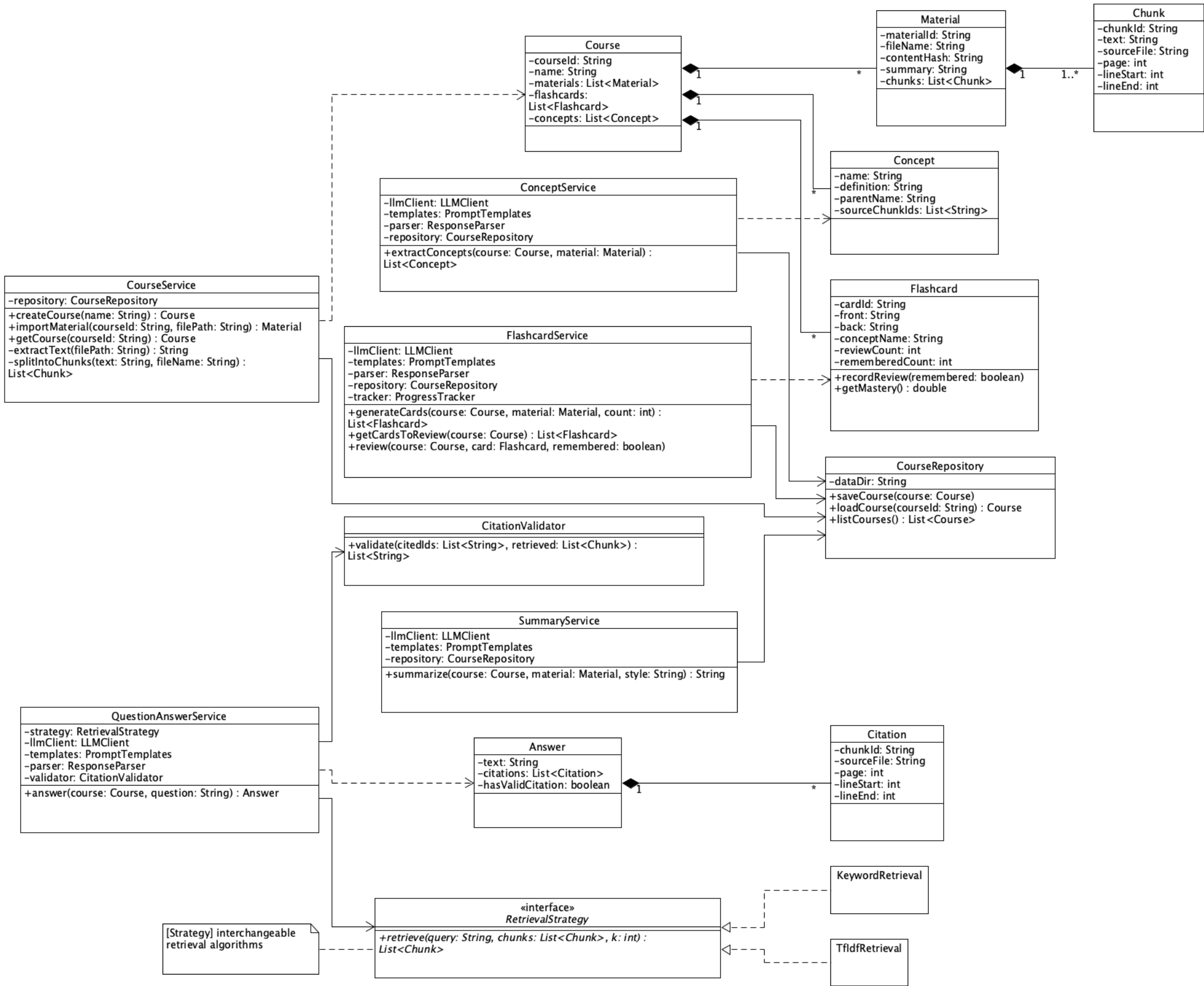
class-view-1 Complete diagram. Use zoom to read detailed labels.

View 2: LLM access and prompts



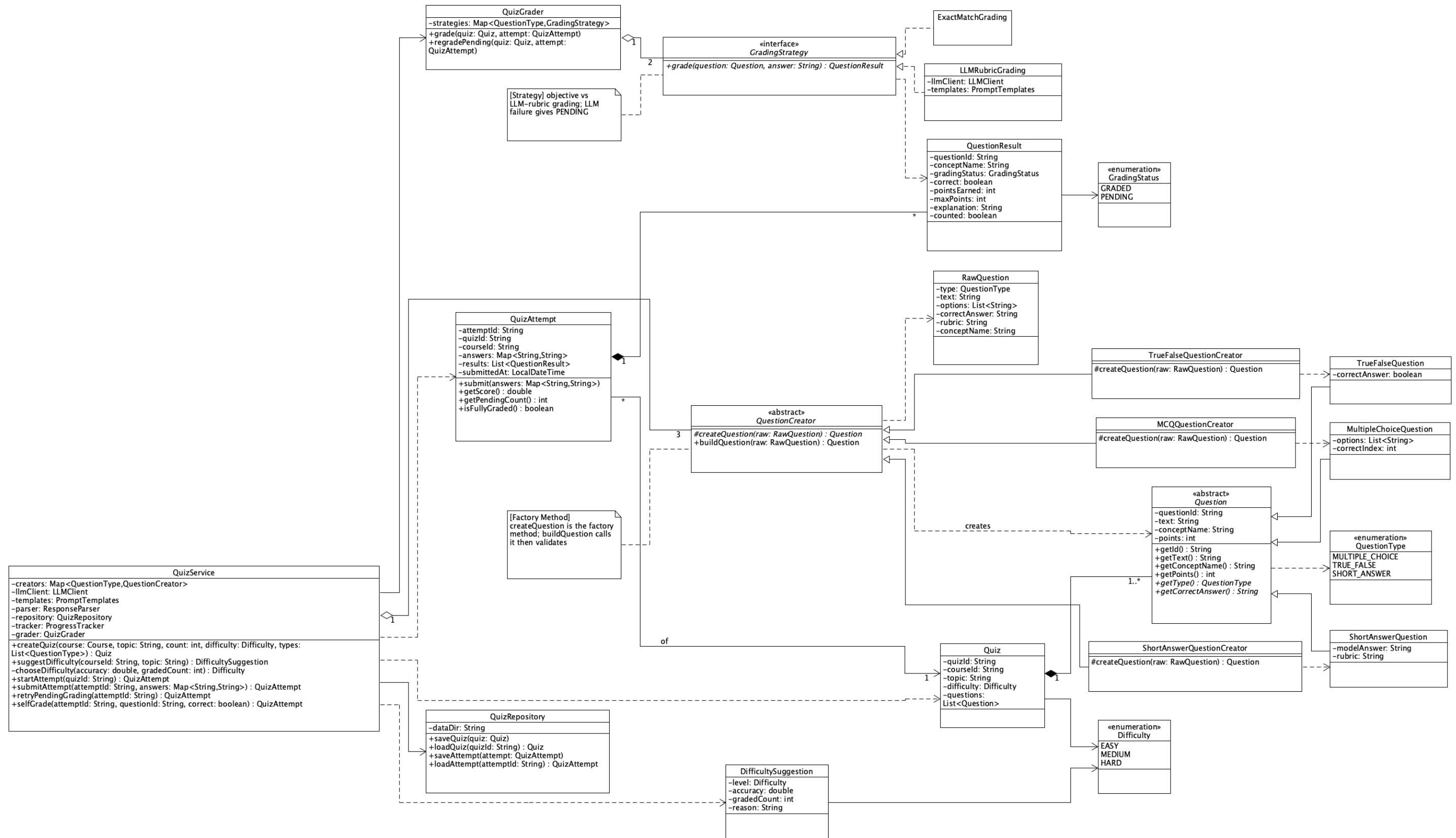
class-view-2 Complete diagram. Use zoom to read detailed labels.

View 3: Courses, materials, concepts, Q&A and flashcards (F01, F05)



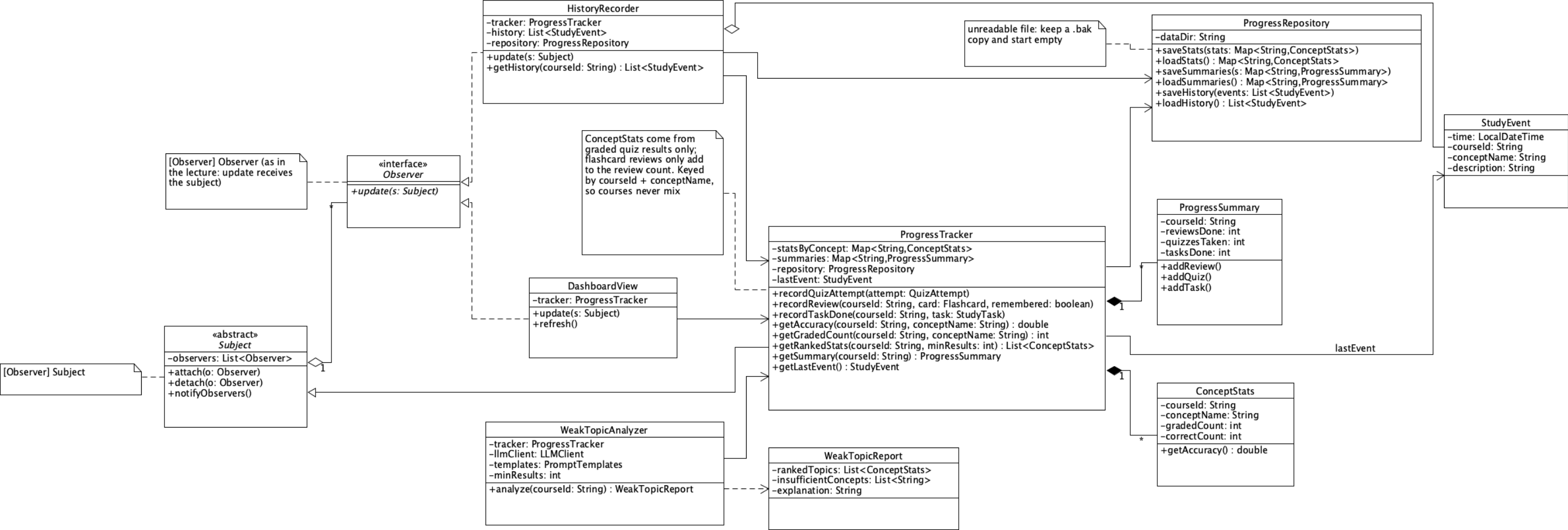
class-view-3 Complete diagram. Use zoom to read detailed labels.

View 4: Quizzes, attempts and grading (F06, F07, F08)



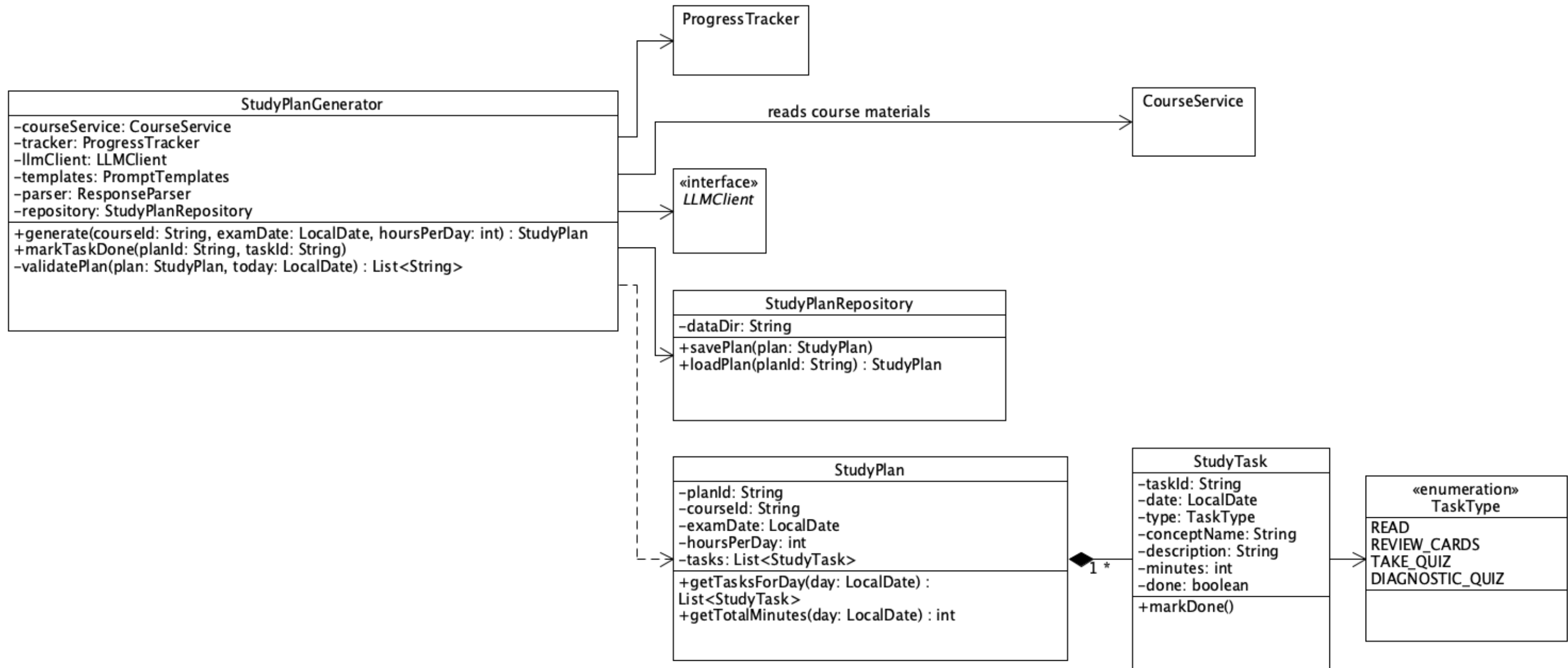
class-view-4 Complete diagram. Use zoom to read detailed labels.

View 5: Progress, weak topics and events (F09, F11)



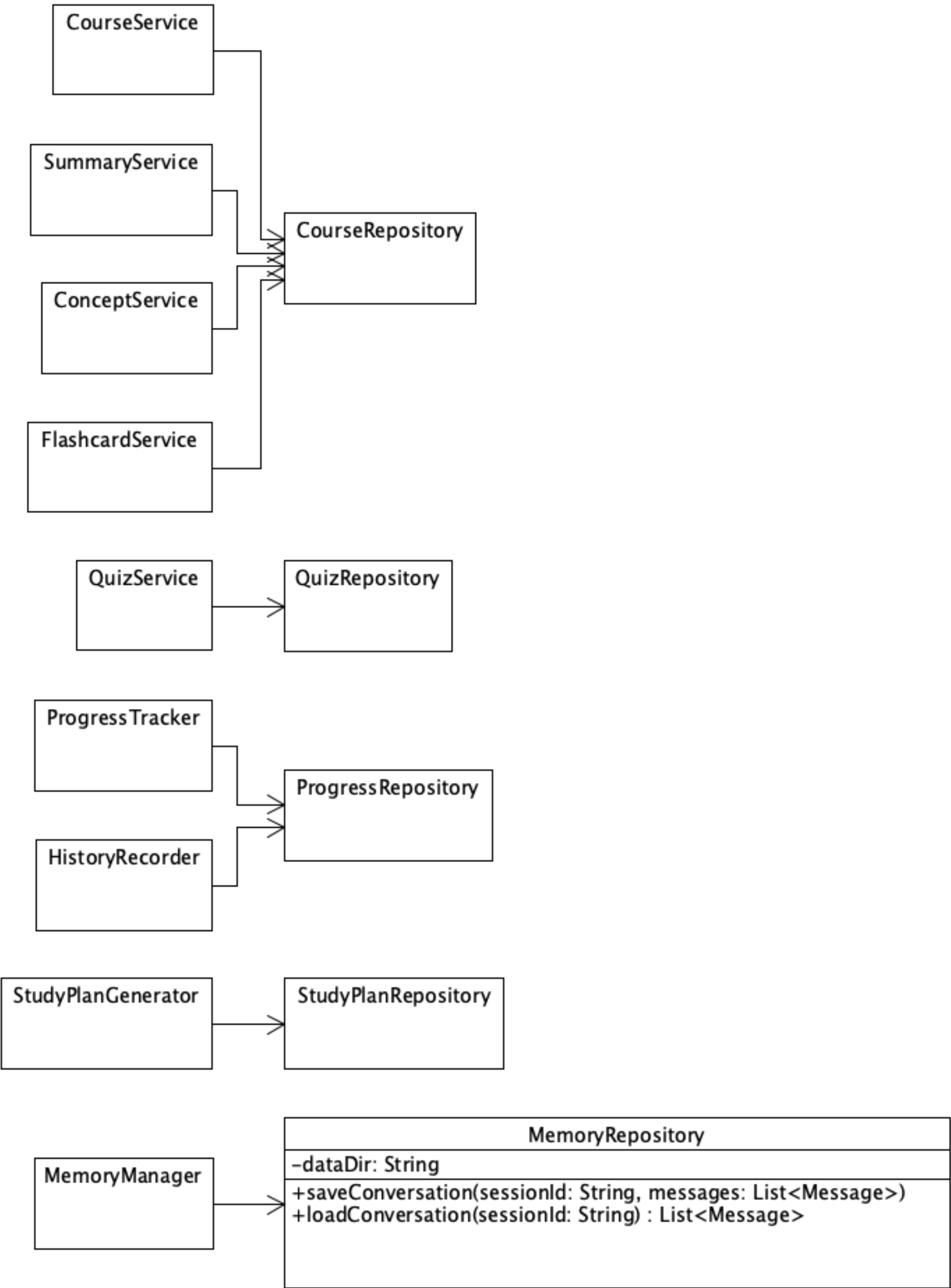
class-view-5 Complete diagram. Use zoom to read detailed labels.

View 6: Study plan (F10)



class-view-6 Complete diagram. Use zoom to read detailed labels.

View 7: Persistence (JSON files)



class-view-7 Complete diagram. Use zoom to read detailed labels.

4 Design pattern explanations

The design uses five patterns: Facade, Strategy, Adapter, Factory Method and Observer.

Facade: AgentFacade

- **Problem:** The Swing GUI and the CLI both need every feature. If they called the subsystems directly, the logic would be duplicated and both interfaces would depend on the internal classes.
- **Classes:** AgentFacade, AgentController, CourseService, QuizService, FlashcardService, StudyPlanGenerator, ProgressTracker, SwingGUI, CLI.
- **Roles:** AgentFacade is the Facade and gives one simple entry point. AgentController, CourseService, QuizService, FlashcardService, StudyPlanGenerator and ProgressTracker are the subsystems that do the work. SwingGUI and CLI are the clients that only call AgentFacade.
- **Why appropriate:** Each feature is written once and both interfaces behave the same. The subsystems can change without changing the UI code.
- **Without it:** The GUI and the CLI would each connect to the subsystems separately, causing duplicate code and inconsistent behavior.

Strategy: Retrieval and Grading

- **Problem:** Two places need interchangeable algorithms: finding the relevant chunks for a question, and grading an answer.
- **Classes:** RetrievalStrategy, KeywordRetrieval, TfIdfRetrieval, QuestionAnswerService, GradingStrategy, ExactMatchGrading, LLMRubricGrading, QuizGrader.
- **Roles:** RetrievalStrategy and GradingStrategy are the Strategy interfaces. KeywordRetrieval and TfIdfRetrieval are two retrieval algorithms. ExactMatchGrading grades multiple-choice and true/false questions, and LLMRubricGrading grades short answers. QuestionAnswerService and QuizGrader are the contexts that use a strategy.
- **Why appropriate:** A new retrieval or grading algorithm is added as a new class without changing the classes that use it.
- **Without it:** QuestionAnswerService and QuizGrader would contain large if/else blocks that grow with every new algorithm.

Adapter: LLM provider

- **Problem:** The OpenAI API has its own request and response format, but the agent should depend only on a small interface of its own.
- **Classes:** LLMClient, OpenAIAdapter, OpenAIApiClient.
- **Roles:** LLMClient is the Target interface used by the rest of the program. OpenAIApiClient is the Adaptee that talks to the OpenAI API. OpenAIAdapter implements LLMClient and converts between the two.
- **Why appropriate:** AgentPlanner, the tools and the services depend on LLMClient and not on OpenAI, so changing the provider only needs a new adapter.
- **Without it:** OpenAI-specific code would be spread through the planner, the tools and the services, making them hard to test and to change.

Factory Method: QuestionCreator

- **Problem:** Quiz questions have different types with different structure and validation rules, and they are built from LLM output. QuizService should not need to know how each type is constructed.
- **Classes:** QuestionCreator, MCQQuestionCreator, TrueFalseQuestionCreator, ShortAnswerQuestionCreator, Question, MultipleChoiceQuestion, TrueFalseQuestion, ShortAnswerQuestion, QuizService.

- **Roles:** QuestionCreator is the abstract Creator. It declares the factory method createQuestion and calls it in buildQuestion, which also does the common validation. The three concrete creators each create one type of question. Question is the abstract Product, and MultipleChoiceQuestion, TrueFalseQuestion and ShortAnswerQuestion are the concrete products. QuizService is the client that calls buildQuestion.
- **Why appropriate:** A new question type only needs a new creator and a new product, and the type-specific validation stays with its creator. QuizService does not change.
- **Without it:** QuizService would need a switch on the question type that must be edited for every new type.

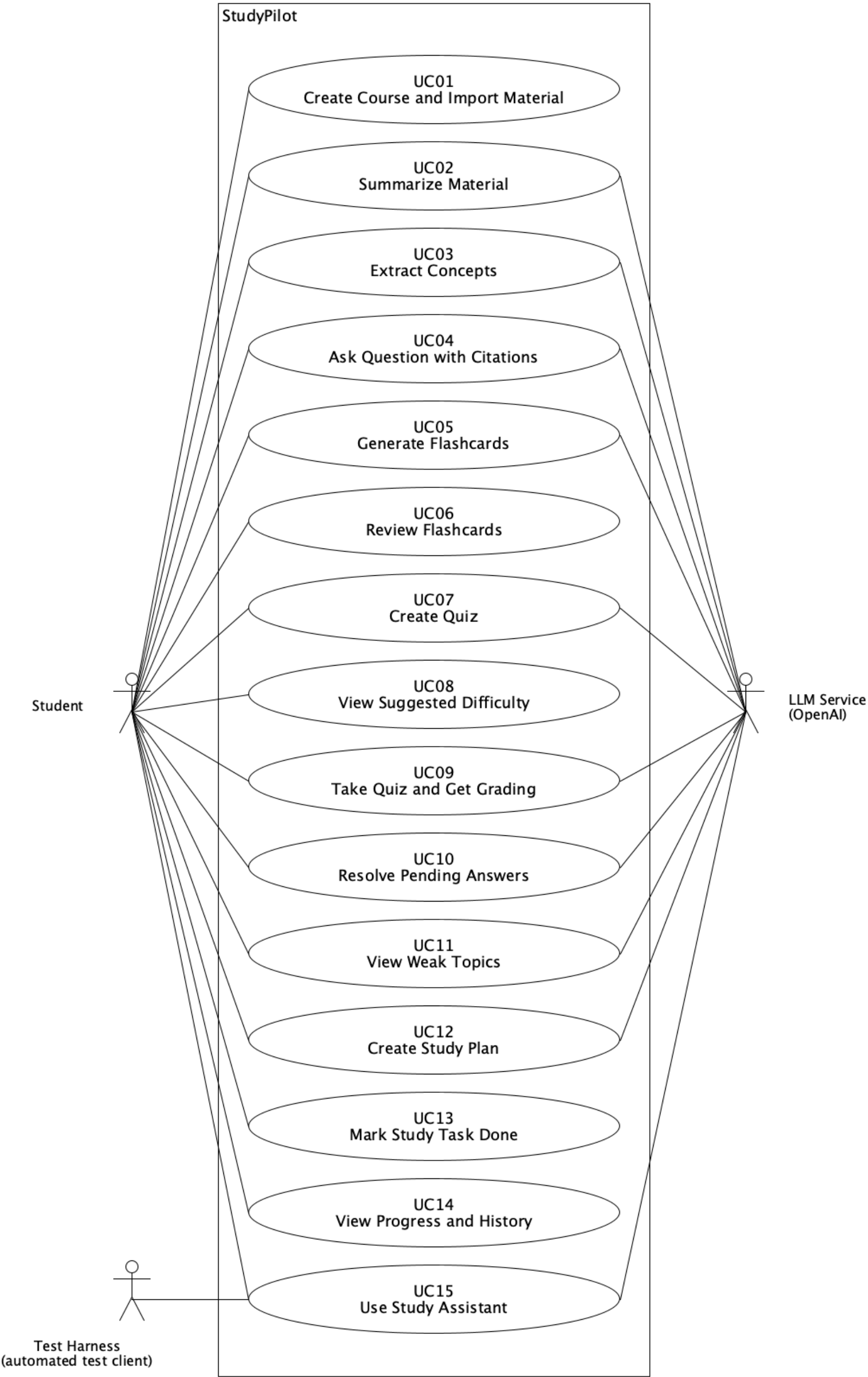
Observer: Study progress

- **Problem:** The dashboard and the session history must update when the student finishes a review, a quiz or a plan task, without the progress code knowing about them.
- **Classes:** Subject, Observer, ProgressTracker, DashboardView, HistoryRecorder.
- **Roles:** Subject keeps the list of observers and can notify them. ProgressTracker is the concrete subject and notifies the observers after it records a result. Observer is the interface with the update method. DashboardView and HistoryRecorder are the concrete observers: they read the tracker and refresh the screen or save the history.
- **Why appropriate:** A new listener can be added without changing ProgressTracker.
- **Without it:** The quiz and flashcard code would have to call the dashboard and the history code directly.

5 Use case diagram

Actor	Type	Description
Student	Primary	Uses the Swing GUI or the CLI. Both call AgentFacade.
Test Harness (automated test client)	Primary (CLI only)	Invokes UC15 through the CLI with --json. Other CLI commands can prepare course data for a test.
LLM Service (OpenAI)	Secondary	External service reached through LLMClient / OpenAIAdapter. It is called by UC02–UC05, UC07, UC09, UC10, UC11, UC12 and UC15.

StudyPilot use case diagram



use-cases Complete diagram. Use zoom to read detailed labels.

6 Detailed use cases

UC01 - Create Course and Import Material

Actor: Student

Goal: Have a course whose study material is stored as searchable chunks with their source location.

Preconditions: The application is running. The student can create a course or select an existing one.

Trigger: The student chooses "New Course" and "Add Material" (GUI) or runs course add / material import (CLI).

Main scenario

1. Student enters a course name.
2. AgentFacade.createCourse calls CourseService.createCourse.
3. Student selects a file.
4. AgentFacade.importMaterial calls CourseService.importMaterial.
5. The service checks the file, extracts the text and splits it into Chunks that keep file name and page or line range.
6. CourseRepository.saveCourse stores the course.
7. The system shows the number of pages and chunks.

Alternative and exception flows

- 3a. Unsupported, corrupt or empty file → the system rejects the import; no material is added. A course created earlier remains available.
- 3b. File too large → size-limit message.
- 5a. Same content hash as an existing material → the system warns and skips it.

Postconditions: The course contains the new Material with its Chunks, saved as JSON.

Related features: F01

UC02 - Summarize Material

Actors: Student; LLM Service

Goal: Get a short summary of a material in a chosen style.

Preconditions: The material exists. The LLM Service is reachable.

Trigger: Student selects a material and clicks "Summarize" or runs summarize <materialId> --style bullet.

Main scenario

1. Student chooses the style.
2. AgentFacade.summarize calls SummaryService.summarize.
3. The service gets a prompt from PromptTemplates.summaryPrompt and calls LLMClient.generate.
4. The response is checked and stored in Material.summary.
5. The summary is shown.

Alternative and exception flows

- 3a. LLM error or timeout → retry once, then show an error message.
- 4a. Empty or malformed response → regenerate.
- 3b. Material too long → summarize chunk by chunk and merge the results.

Postconditions: The material has a stored summary.

Related features: F02

UC03 - Extract Concepts

Actors: Student; LLM Service

Goal: Get a list of key concepts with short definitions.

Preconditions: The material exists. The LLM Service is reachable.

Trigger: Student clicks "Extract Concepts" or runs concepts extract <materialId>.

Main scenario

1. AgentFacade.extractConcepts calls ConceptService.extractConcepts.
2. The LLM returns structured JSON.
3. ResponseParser.parseConcepts validates it and returns List<Concept>.
4. Duplicates are merged.
5. The concepts are saved in the course and shown.

Alternative and exception flows

- 3a. Invalid JSON → retry once with a stricter prompt.
- 4a. No concept found → tell the student.
- 4b. A parent link that forms a cycle → drop that link.

Postconditions: The course holds the concept list.

Related features: F03

UC04 - Ask Question with Citations

Actors: Student; LLM Service

Goal: Get an answer based only on the course materials, with citations that point to the source.

Preconditions: The course has at least one imported material. The LLM Service is reachable.

Trigger: Student types a question in the Ask panel or runs ask "<question>" --course <id>.

Main scenario

1. AgentFacade.askQuestion calls QuestionAnswerService.answer.
2. A RetrievalStrategy returns the top-k Chunks.
3. The prompt from PromptTemplates.qaPrompt contains the chunks with their ids.
4. The LLM returns an answer and the cited chunk ids.
5. CitationValidator.validate keeps only ids that are in the retrieved set.
6. The system shows the answer with file name and page/line range for each valid citation.

Alternative and exception flows

2a. No relevant chunk → the system says the materials do not cover it; the LLM is not called.

5a. All cited ids are invalid → the answer is shown as "uncited".

4a. LLM failure → error message, no answer is made up.

1a. Question too vague → the system asks a clarifying question.

Postconditions: No stored data changes.

Related features: F04

UC05 - Generate Flashcards

Actors: Student; LLM Service

Goal: Get flashcards for a material.

Preconditions: The material exists. The LLM Service is reachable.

Trigger: Student clicks "Generate" in Flashcards or runs flashcards generate <materialId> --count 20.

Main scenario

1. Student gives a count.
2. AgentFacade.generateFlashcards calls FlashcardService.generateCards for the chosen material.
3. ResponseParser.parseFlashcards reads the LLM output.
4. Cards with empty front/back and duplicates are removed.
5. The cards are saved in the course with CourseRepository and shown.

Alternative and exception flows

3a. Malformed output → retry once.

4a. Fewer cards than requested → show the real count.

2a. LLM failure → error message.

Postconditions: The course holds the new Flashcards with reviewCount = 0.

Related features: F05

UC06 - Review Flashcards

Actor: Student

Goal: Practise flashcards and record which ones are remembered.

Preconditions: The course has flashcards.

Trigger: Student opens the Review session or runs review <courseId>.

Main scenario

1. FlashcardService.getCardsToReview returns cards never reviewed or with low mastery first.
2. The system shows the front of a card.
3. Student flips it and chooses "Remembered" or "Not yet".
4. AgentFacade.reviewCard(courseId, cardId, remembered) calls FlashcardService.review, which updates reviewCount and rememberedCount and saves the course.
5. ProgressTracker.recordReview publishes a StudyEvent.

6. Steps 2–5 repeat until the student stops or no cards are left.

Alternative and exception flows

1a. No cards to review → show "nothing to review".

4a. Unknown card id → rejected with an error.

Postconditions: Card counts and the progress statistics are updated; dashboard and history show the new event.

Related features: F05, F11

UC07 - Create Quiz

Actors: Student; LLM Service

Goal: Get a quiz with mixed question types for a topic at a suitable difficulty.

Preconditions: The course has materials. The LLM Service is reachable.

Trigger: Student chooses "New Quiz" or runs quiz create --topic <t> --count 10.

Main scenario

1. Student enters topic and number of questions.
2. AgentFacade.suggestDifficulty reads graded statistics through QuizService and shows the suggested difficulty; the student accepts or changes it, and selects the question types.
3. AgentFacade.createQuiz(courseId, topic, count, difficulty, types) calls QuizService.createQuiz.
4. The LLM returns draft questions; ResponseParser.parseRawQuestions makes RawQuestions.
5. For each one the matching QuestionCreator.buildQuestion builds and validates a Question.
6. The Quiz is saved with QuizRepository.saveQuiz and shown.

Alternative and exception flows

- 2a. Student enters an invalid difficulty → ignored, suggestion is used.
- 5a. Invalid question (e.g. no correct option) → discard and ask for a replacement (bounded retries).
- 5b. Not enough material → fewer questions, with a notice.
- 4a. LLM failure → error message.

Postconditions: An immutable Quiz is stored.

Related features: F06, F07

UC08 - View Suggested Difficulty

Actor: Student

Goal: See which difficulty is suggested for a topic and why.

Preconditions: A course and topic exist.

Trigger: Student requests a suggestion in the Quiz screen or runs quiz difficulty --topic <t>.

Main scenario

1. AgentFacade.suggestDifficulty calls QuizService.suggestDifficulty.
2. ProgressTracker.getAccuracy(courseId, topic) and getGradedCount(courseId, topic) give the numbers.
3. QuizService.chooseDifficulty returns EASY, MEDIUM or HARD.

4. The system shows the level with a reason such as "accuracy 85% over 12 questions".

Alternative and exception flows

2a. Too few graded results → default MEDIUM, and the reason says so.

Postconditions: No stored data changes.

Related features: F07

UC09 - Take Quiz and Get Grading

Actors: Student; LLM Service (only for short answers)

Goal: Answer a quiz and get a score with explanations.

Preconditions: A Quiz exists.

Trigger: Student clicks "Take Quiz" or runs quiz take <quizId>.

Main scenario

1. QuizService.startAttempt creates a new QuizAttempt with a new attemptId.
2. Student answers the questions and submits.
3. AgentFacade.submitAttempt calls QuizService.submitAttempt.
4. QuizGrader.grade chooses a GradingStrategy per question type: ExactMatchGrading for multiple choice / true-false, LLMRubricGrading for short answers.
5. Each QuestionResult is GRADED; wrong answers get an explanation.
6. The attempt is saved.
7. ProgressTracker.recordQuizAttempt records graded results with counted = false, saves progress, marks them counted and publishes a StudyEvent; QuizService saves the attempt again to persist the flags.
8. The system shows score over graded points, and "N pending" if there are any.

Alternative and exception flows

2a. Unanswered question → scored 0 and flagged.

4a. LLM grader fails or returns invalid output → that result is PENDING; the student can resolve it now or later with UC10.

3a. The same attemptId is submitted again → the stored result is returned, nothing is graded or counted twice.

Postconditions: A saved QuizAttempt; progress statistics include its graded results.

Related features: F08, F11

UC10 - Resolve Pending Answers

Actors: Student; LLM Service (only for automatic re-grading)

Goal: Turn PENDING results of a saved attempt into graded results.

Preconditions: A saved attempt has at least one PENDING result.

Trigger: Student opens the saved attempt and clicks "Retry grading" or "Self-grade" (CLI: quiz regrade <attemptId>, quiz selfgrade <attemptId> <questionId> <right|wrong>).

Main scenario

1. Student chooses "Retry grading".

2. AgentFacade.retryPendingGrading calls QuizService.retryPendingGrading, which calls QuizGrader.regradePending.
3. The results become GRADED.
4. The attempt is saved; ProgressTracker.recordQuizAttempt records newly graded, uncounted results, saves progress, sets their flags and publishes a StudyEvent; QuizService saves the attempt again with the updated flags.
5. The system shows the updated score.

Alternative and exception flows

- 1a. Student prefers to self-grade: the system shows the model answer and the student marks it right or wrong (AgentFacade.selfGrade → QuizService.selfGrade); continue at step 4.
- 2a. LLM fails again → the result stays PENDING; the student may try again later.
- 1b. The attempt has no pending result → nothing to do, the system says so.

Postconditions: Resolved results are GRADED and now count in weak-topic and difficulty statistics.

Related features: F08, F11

UC11 - View Weak Topics

Actors: Student; LLM Service

Goal: See which topics need more study and why.

Preconditions: The course exists.

Trigger: Student opens Progress → Weak Topics or runs weak-topics --course <id>.

Main scenario

1. AgentFacade.getWeakTopics calls WeakTopicAnalyzer.analyze.
2. ProgressTracker.getRankedStats(courseId, minResults) returns concepts with enough graded results, lowest accuracy first.
3. The LLM writes a short explanation and suggestions (PromptTemplates.weakTopicPrompt).
4. The system shows the ranked list with accuracy and number of results, and the explanation.

Alternative and exception flows

- 2a. A concept has fewer than the minimum number of graded results → listed as "insufficient data", not ranked.
- 2b. No data at all → the report says there is not enough data; no "weakest topic" is claimed.
- 3a. LLM failure → the ranking is shown without the explanation.

Postconditions: No stored data changes.

Related features: F09

UC12 - Create Study Plan

Actors: Student; LLM Service

Goal: Get a day-by-day revision schedule until the exam.

Preconditions: The course has materials.

Trigger: Student chooses "New Plan" or runs `plan --exam <date> --hours 2`.

Main scenario

1. Student enters exam date and hours per day.
2. `AgentFacade.createStudyPlan` calls `StudyPlanGenerator.generate`.
3. The generator gets the course materials through `CourseService.getCourse` and reads `ProgressTracker.getRankedStats(courseId, minResults)` to find the weak topics.
4. The LLM proposes `StudyTasks` using a prompt containing both sources (`PromptTemplates.studyPlanPrompt`, `ResponseParser.parseStudyTasks`).
5. `StudyPlanGenerator.validatePlan` checks the plan (dates in range, daily minutes within the limit).
6. If rules are broken, the generator repairs or regenerates (bounded retries).
7. `StudyPlanRepository.savePlan` stores the `StudyPlan` and the system shows it.

Alternative and exception flows

- 1a. Exam date in the past → rejected.
- 5a. Time budget too small → warn and propose a reduced scope.
- 3a. No learning history → no weak topic is claimed; the plan starts with a diagnostic quiz or asks the student to pick topics.
- 6a. Still invalid after the retries → show an error.

Postconditions: A valid `StudyPlan` is stored.

Related features: F10

UC13 - Mark Study Task Done

Actor: Student

Goal: Record that a planned task was completed.

Preconditions: A `StudyPlan` exists.

Trigger: Student ticks a task in the plan view or runs `plan complete <planId> <taskId>`.

Main scenario

1. `AgentFacade.completeTask(planId, taskId)` calls `StudyPlanGenerator.markTaskDone`.
2. The generator calls `StudyTask.markDone` and saves the plan with `StudyPlanRepository`.
3. `ProgressTracker.recordTaskDone(courseId, task)` updates the counts and publishes a `StudyEvent`.
4. The dashboard refreshes.

Alternative and exception flows

- 1a. Task already done → nothing changes.
- 2a. Saving fails → error message; the task stays not done.

Postconditions: The task is done; progress counts include it.

Related features: F10, F11

UC14 - View Progress and History

Actor: Student

Goal: See how much was studied and what happened in earlier sessions.

Preconditions: The application is running.

Trigger: Student opens the Dashboard or runs progress / history.

Main scenario

1. AgentFacade.getProgress calls ProgressTracker.getSummary.
2. The system shows the counts.
3. AgentFacade.getHistory(courseId) returns the past StudyEvents from HistoryRecorder, which loads them from ProgressRepository, so earlier sessions are included.
4. While the dashboard is open, ProgressTracker calls notifyObservers() after each new result and DashboardView.update(subject) refreshes the screen.

Alternative and exception flows

- 1a. Nothing studied yet → show zero counts and an empty history.
- 1b. Unreadable progress or history file → a .bak copy is kept, the data starts empty, and the system warns.

Postconditions: No stored data changes.

Related features: F11

UC15 - Use Study Assistant

Actors: Student or Test Harness; LLM Service

Goal: Complete a free-form study request by letting the agent plan and run the right tools.

Preconditions: At least one course exists. The LLM Service is reachable.

Trigger: Student types a request in the Assistant panel, or a script runs agent "<request>" --session <id> --json.

Main scenario

1. AgentFacade.runAgent calls AgentController.handleRequest.
2. MemoryManager.loadSession(sessionId) loads the session and getHistory() gives its earlier messages.
3. AgentPlanner.createPlan asks the LLM for an ExecutionPlan.
4. The controller checks the plan: every tool exists via ToolManager.hasTool, arguments are valid, and there are at most MAX_STEPS steps.
5. For each PlanStep, ToolManager.execute runs the tool and returns a ToolResult.
6. AgentController.decide returns a Decision; on CONTINUE the next step runs.
7. Each step and decision is added to the trace list.
8. The final answer is made from the tool results. The student's request and the assistant's reply are both saved as Messages with MemoryManager.addMessage, and the answer is returned as an AgentResult.

Alternative and exception flows

- 3a. LLM failure or unparseable plan → ask again once, then ABORT with an honest error message.

4a. Unknown tool or invalid arguments → the step is not executed; REPLAN.

6a. FAILED result → RETRY the step (max 2 per step).

6b. Still failing → REPLAN (max 2 per request) with the results so far.

6c. INSUFFICIENT_DATA → CLARIFY: ask the student a question, or REPLAN into a diagnostic quiz.

6d. Limits exceeded → ABORT and report the partial result.

Postconditions: The conversation is saved; the trace is available; the stored data changed only through the tools that ran.

Related features: F12

7 Sequence diagrams

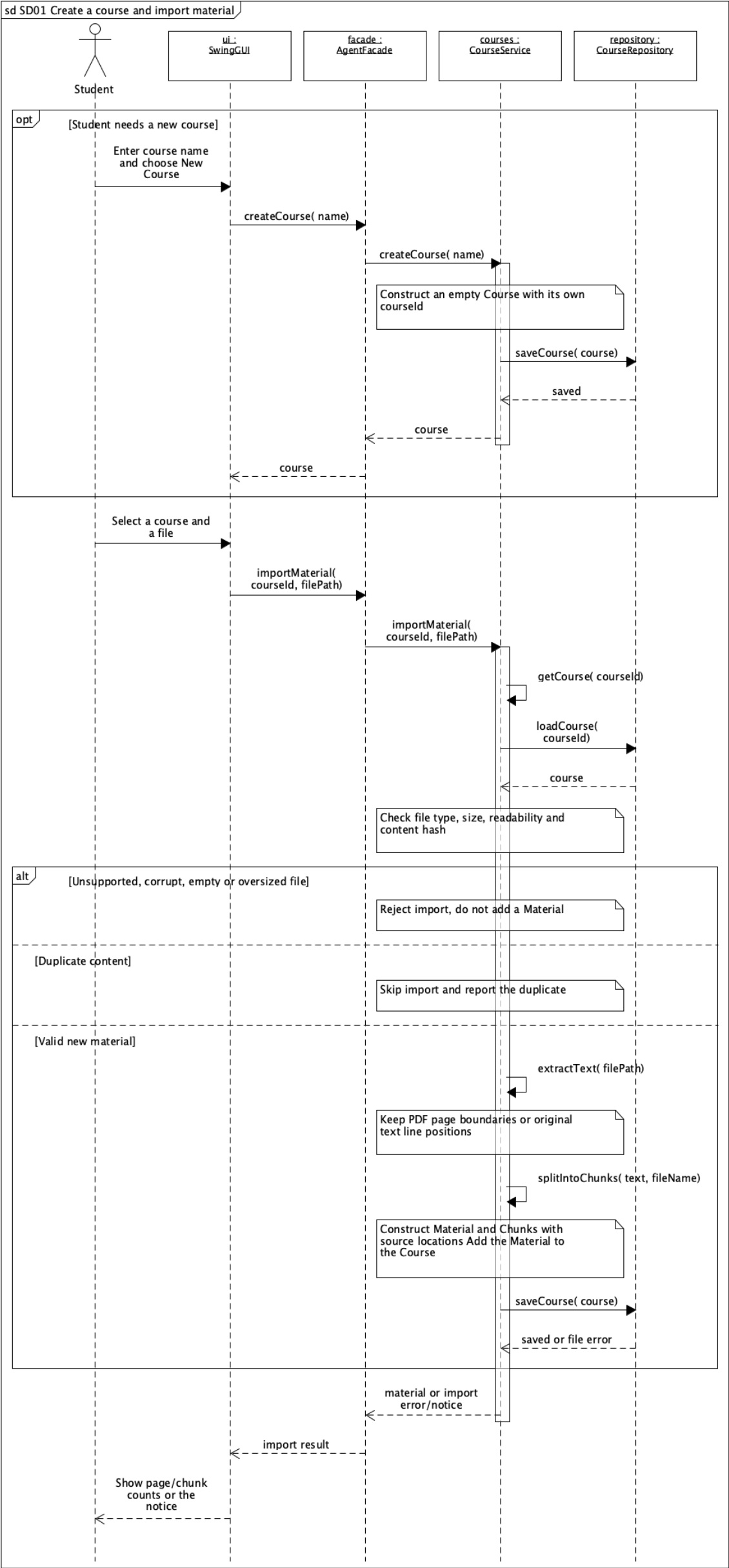
Objects are written as name : Class and underlined. Solid filled arrows are synchronous calls, dashed arrows are returns, and activation bars show a running operation. alt, opt and loop frames show alternatives, optional work and repetition.

Most diagrams use SwingGUI. The CLI shares AgentFacade; SD13 includes the Test Harness and JSON output. SD02 shows OpenAIAdapter calling OpenAIApiClient and the external OpenAI service. Elsewhere llm : LLMClient refers to that adapter.

Diagram	Main interaction	Features	Use cases
SD01	Create a course and import material	F01	UC01
SD02	Summarize material, including the OpenAI Adapter	F02	UC02
SD03	Extract concepts or generate flashcards	F03, F05	UC03, UC05
SD04	Answer a question with checked citations	F04	UC04
SD05	Review a flashcard and notify observers	F05, F11	UC06
SD06	Suggest difficulty and create a quiz	F06, F07	UC07, UC08
SD07	Start and grade a quiz attempt	F08, F11	UC09
SD08	Retry or self-grade pending answers	F08, F11	UC10
SD09	Identify weak topics	F09	UC11
SD10	Generate and validate a study plan	F10	UC12
SD11	Complete a study task	F10, F11	UC13
SD12	Read progress and session history	F11	UC14
SD13	Agent planning, conversation memory and the CLI result	F12	UC15
SD14	Agent tool execution, retry, replan and clarification	F12	UC15

SD01 - Create a course and import material

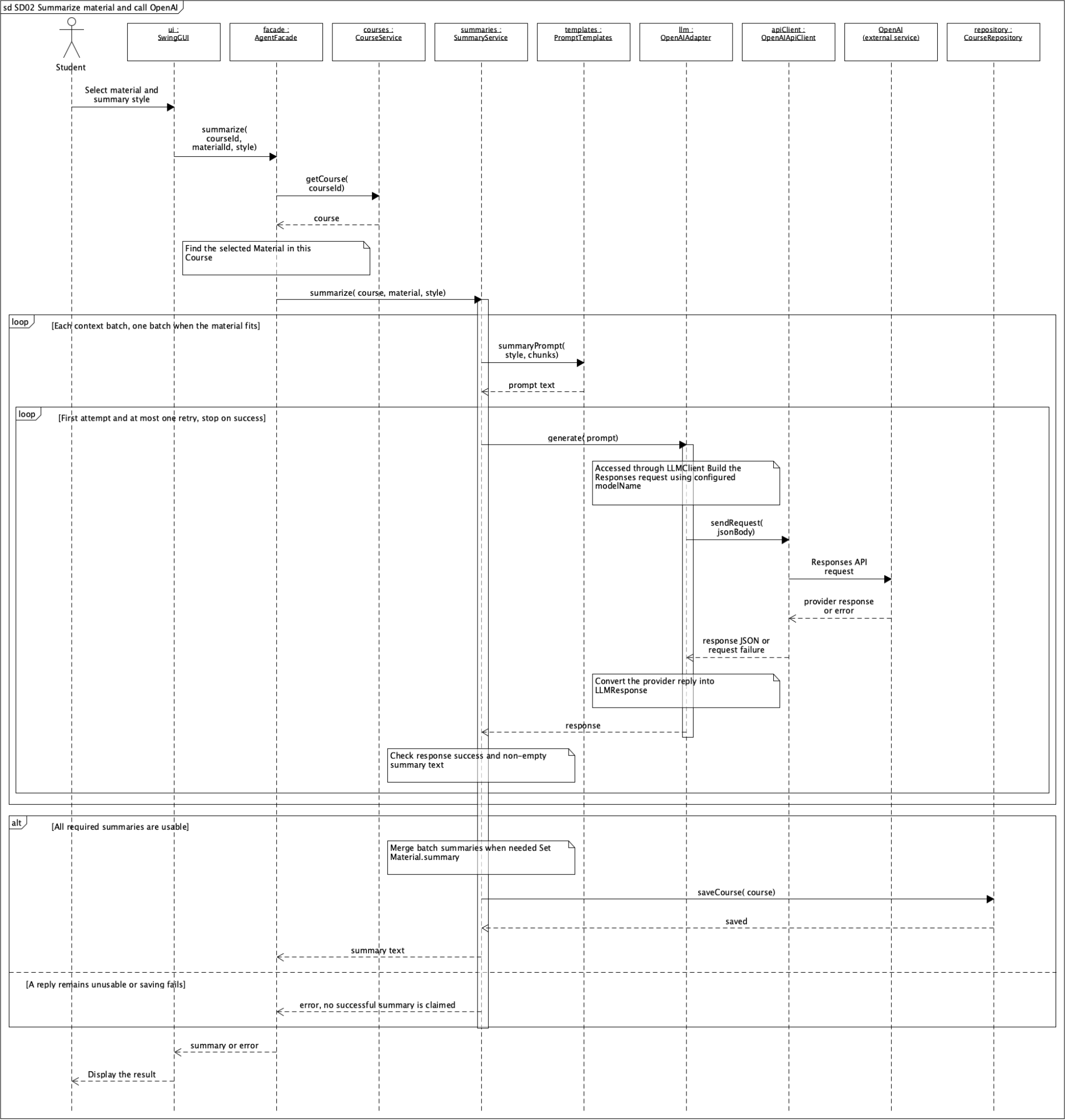
Related feature: F01. Related use case: UC01. Pattern: Facade.



SD01 Complete diagram. Use zoom to read detailed labels.

SD02 - Summarize material and call OpenAI

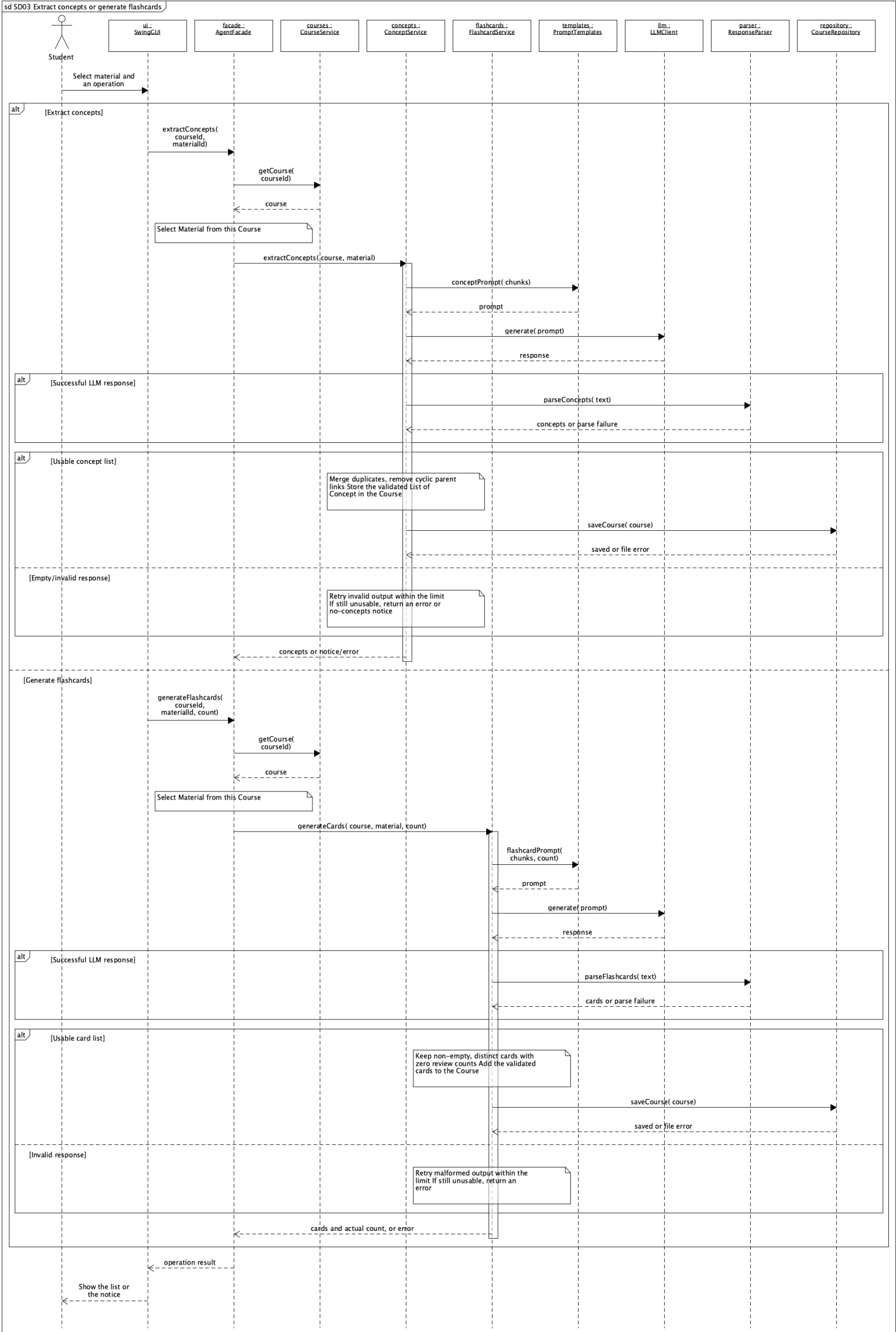
Related feature: F02. Related use case: UC02. Patterns: Facade, Adapter.



SD02 Complete diagram. Use zoom to read detailed labels.

SD03 - Extract concepts or generate flashcards

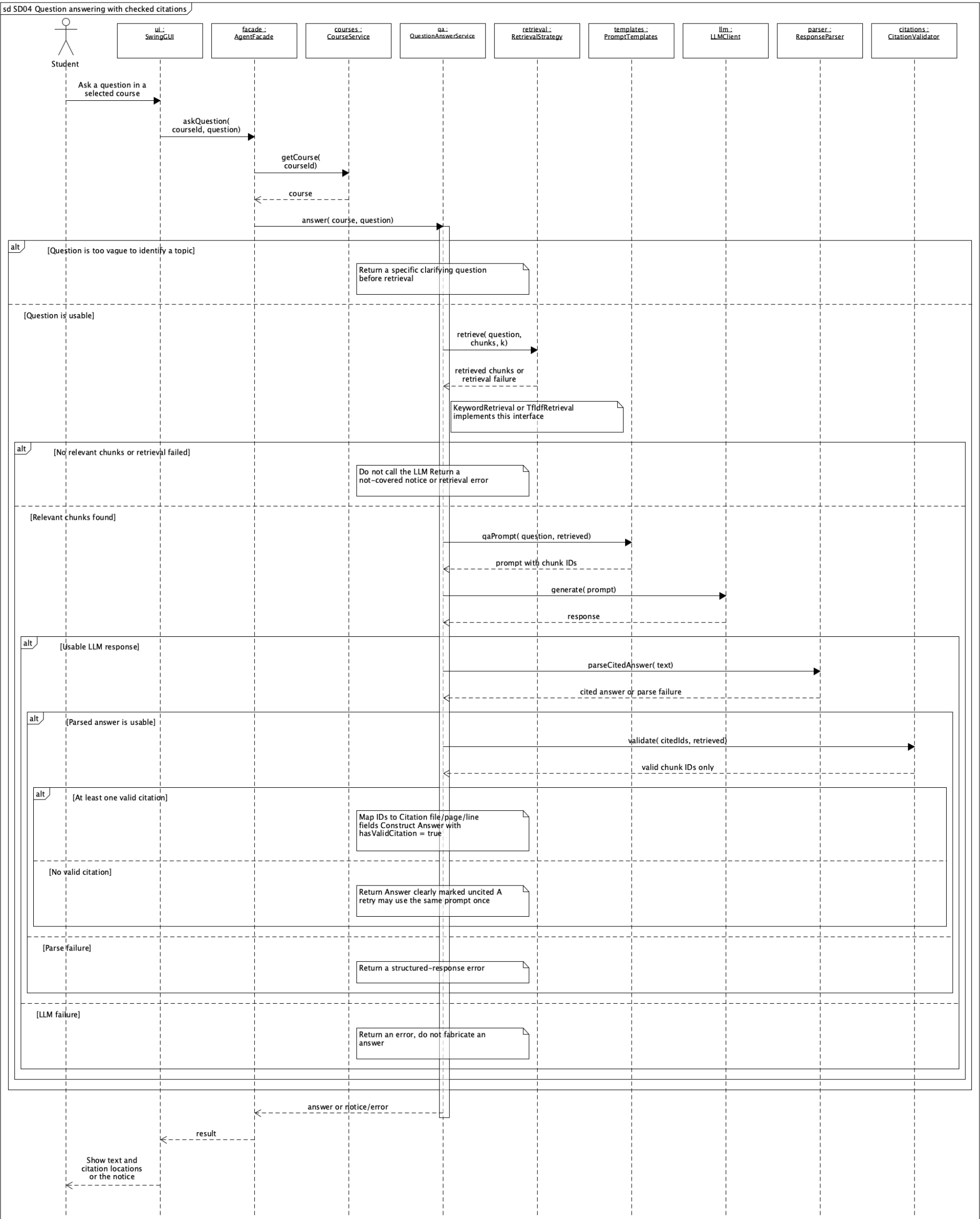
Related features: F03, F05. Related use cases: UC03, UC05. Patterns: Facade, Adapter.



SD03 Complete diagram. Use zoom to read detailed labels.

SD04 - Question answering with checked citations

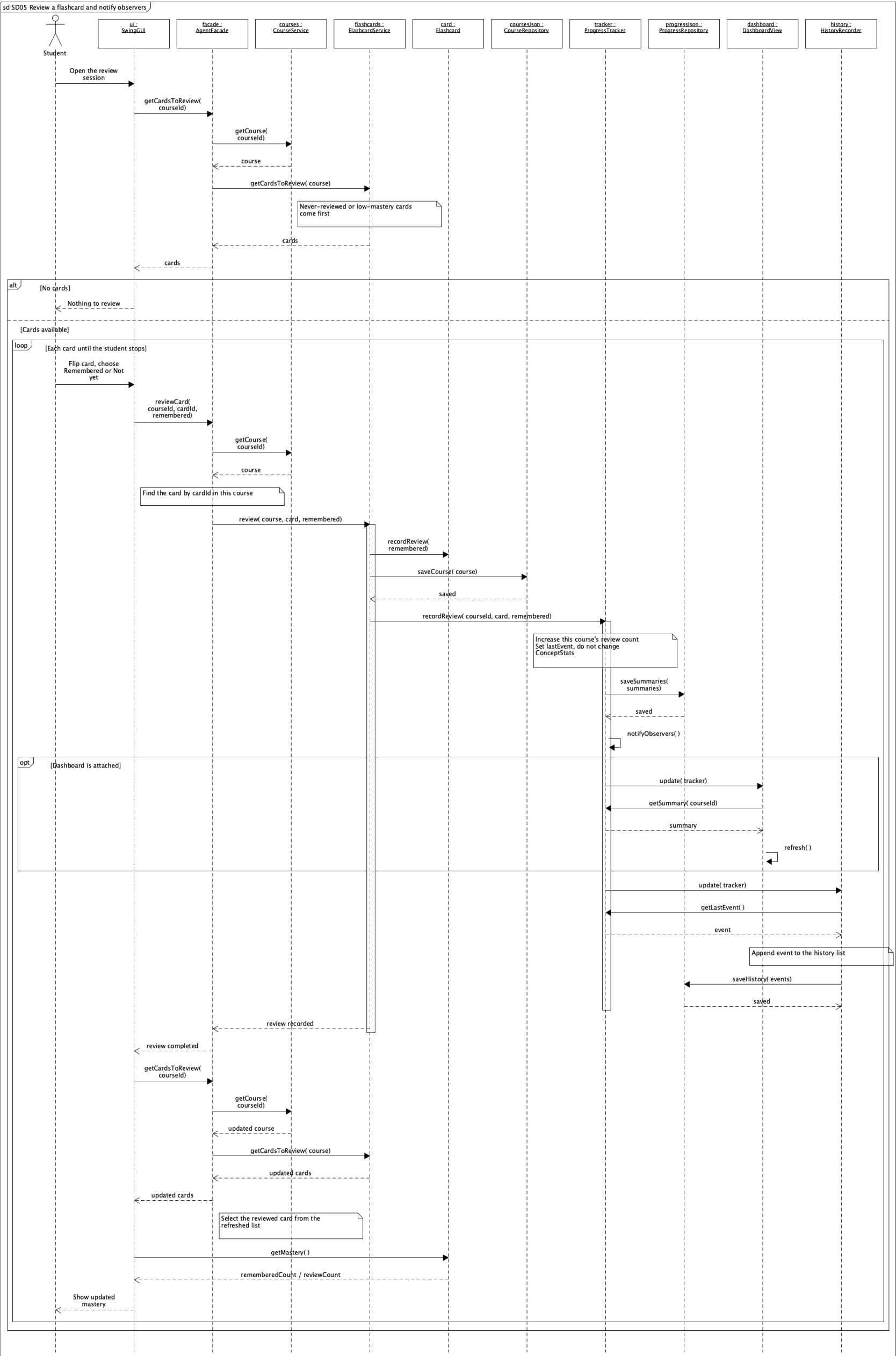
Related feature: F04. Related use case: UC04. Patterns: Facade, Strategy, Adapter.



SD04 Complete diagram. Use zoom to read detailed labels.

SD05 - Review a flashcard and notify observers

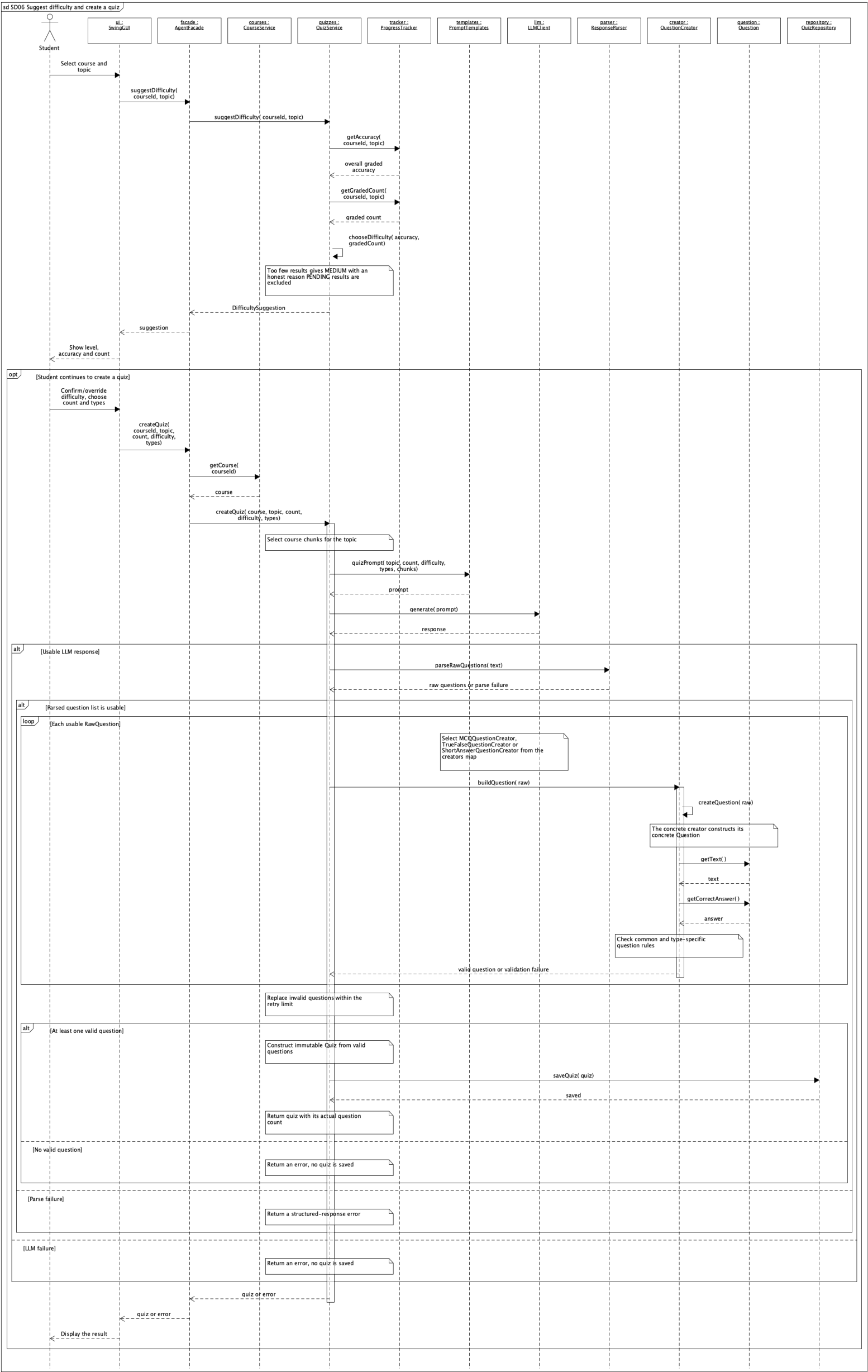
Related features: F05, F11. Related use case: UC06. Patterns: Facade, Observer.



SD05 Complete diagram. Use zoom to read detailed labels.

SD06 - Suggest difficulty and create a quiz

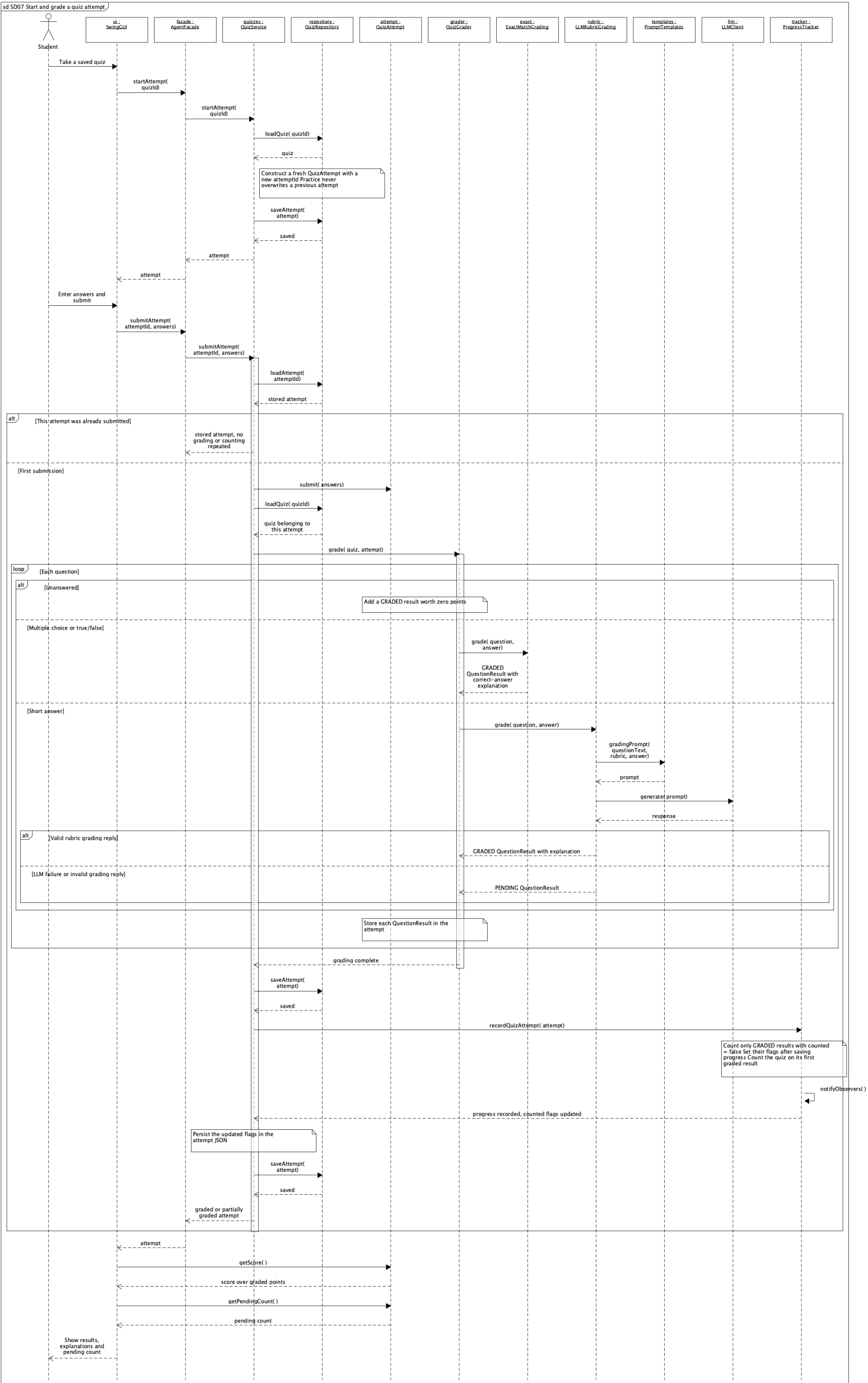
Related features: F06, F07. Related use cases: UC07, UC08. Patterns: Facade, Adapter, Factory Method.



SD06 Complete diagram. Use zoom to read detailed labels.

SD07 - Start and grade a quiz attempt

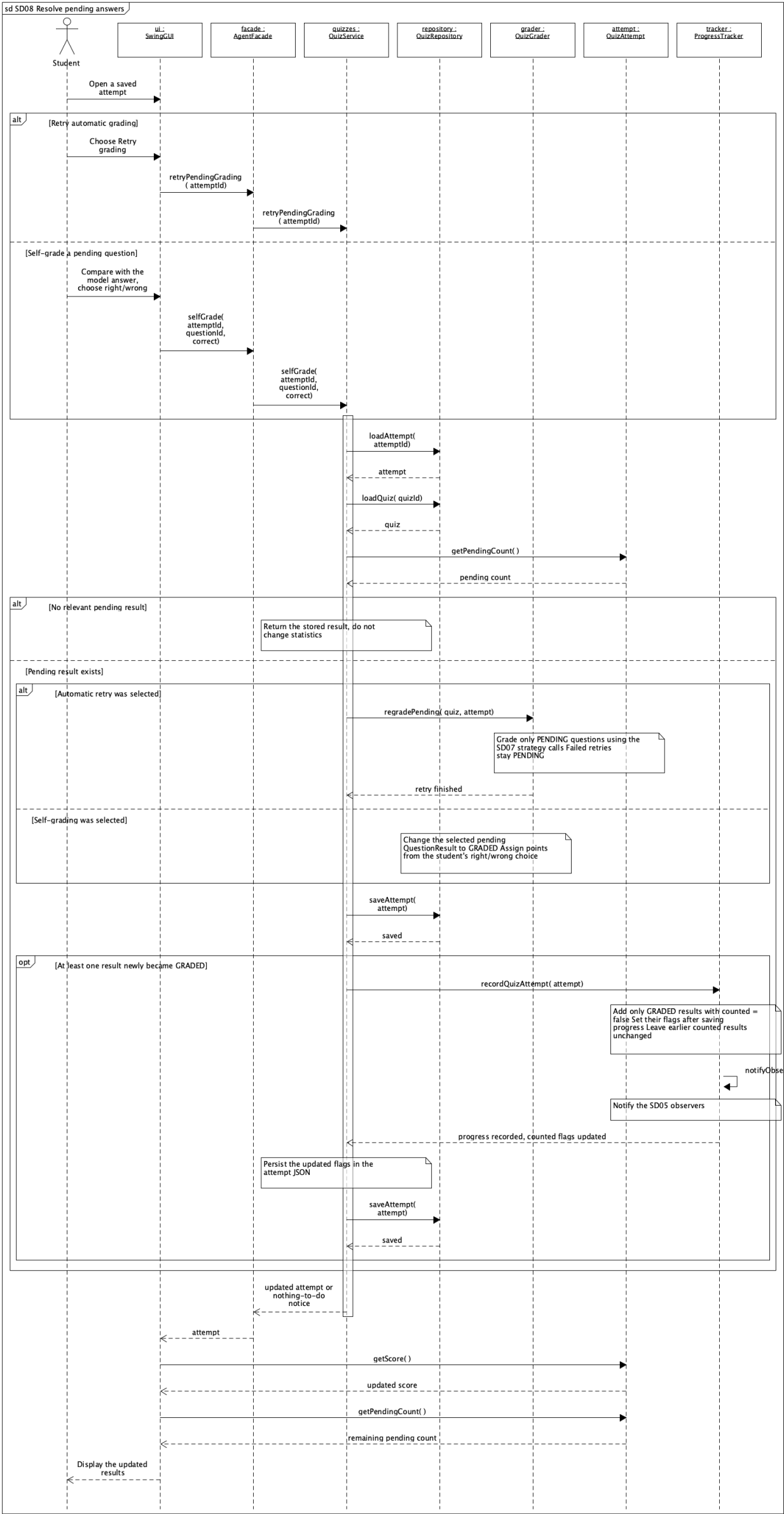
Related features: F08, F11. Related use case: UC09. Patterns: Facade, Strategy, Adapter, Observer.



SD07 Complete diagram. Use zoom to read detailed labels.

SD08 - Resolve pending answers

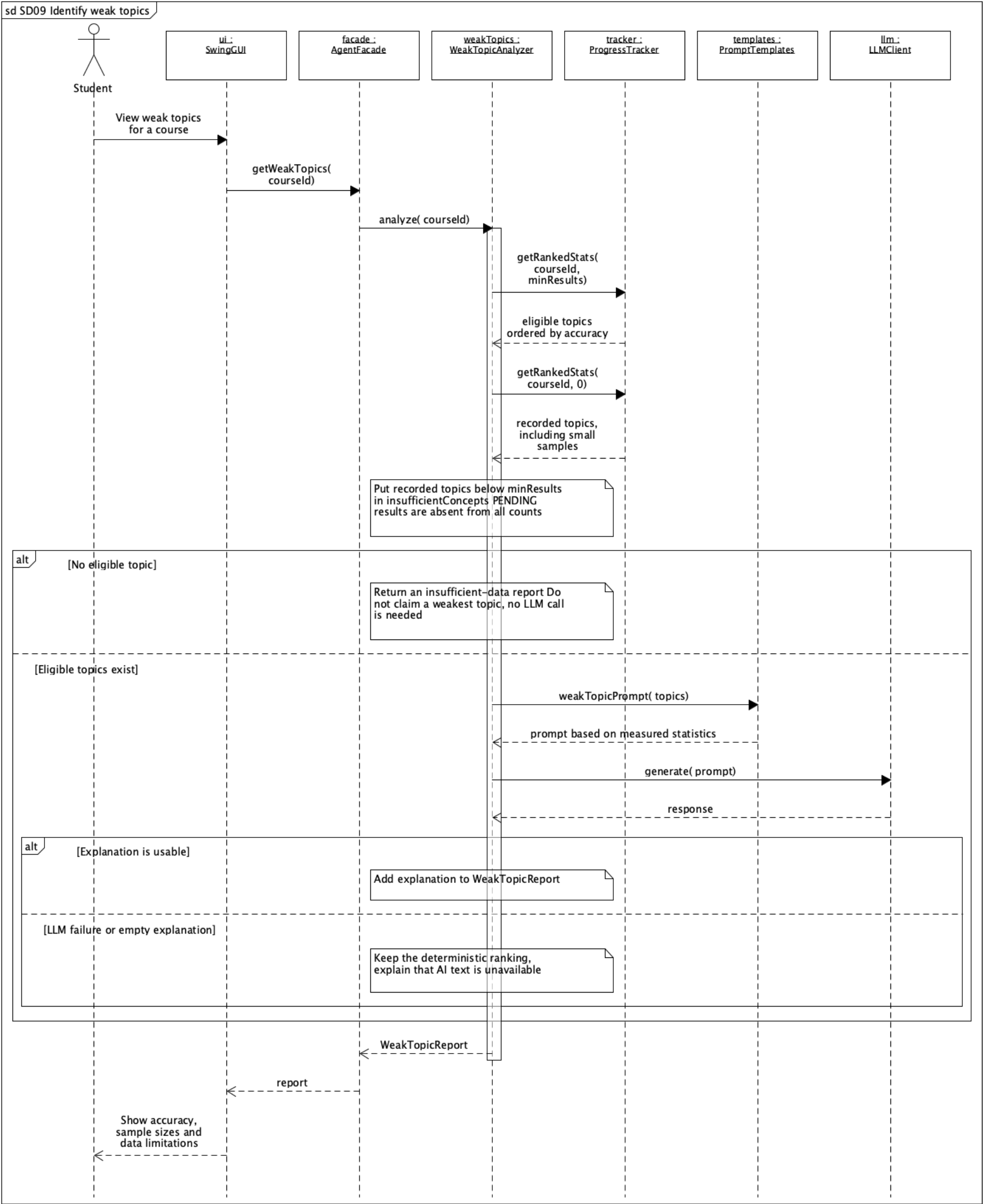
Related features: F08, F11. Related use case: UC10. Patterns: Facade, Strategy, Adapter, Observer.



SD08 Complete diagram. Use zoom to read detailed labels.

SD09 - Identify weak topics

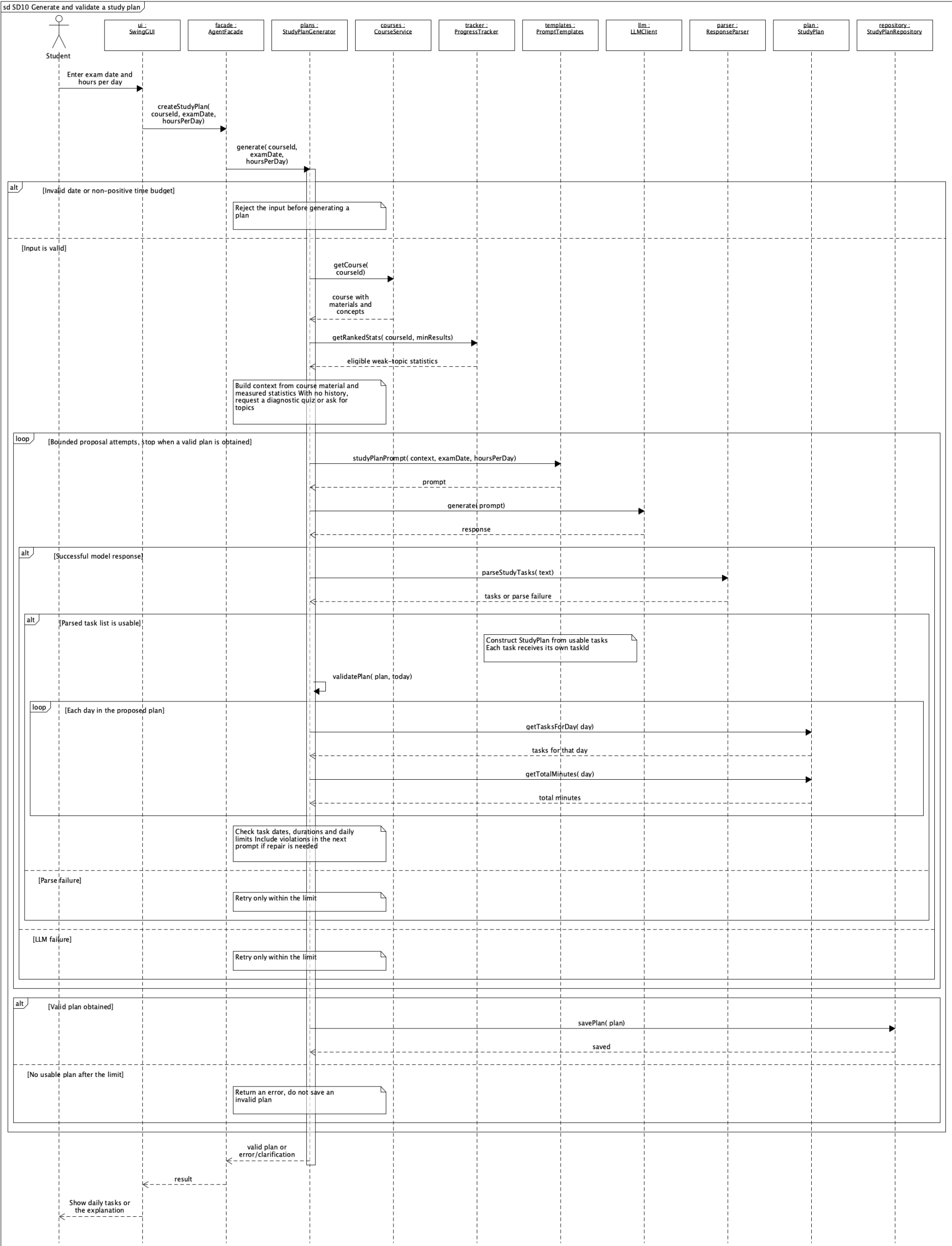
Related feature: F09. Related use case: UC11. Patterns: Facade, Adapter.



SD09 Complete diagram. Use zoom to read detailed labels.

SD10 - Generate and validate a study plan

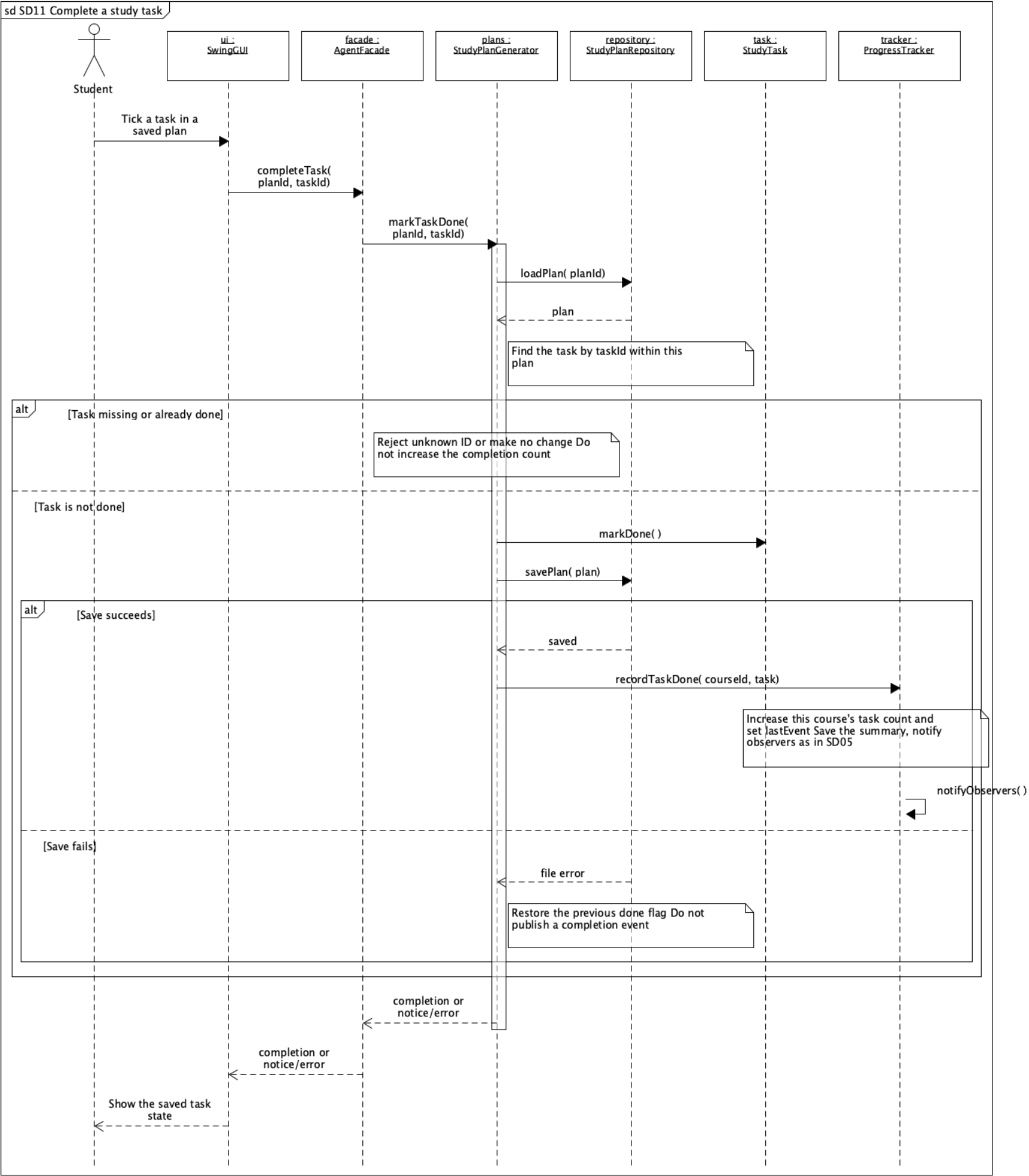
Related feature: F10. Related use case: UC12. Patterns: Facade, Adapter.



SD10 Complete diagram. Use zoom to read detailed labels.

SD11 - Complete a study task

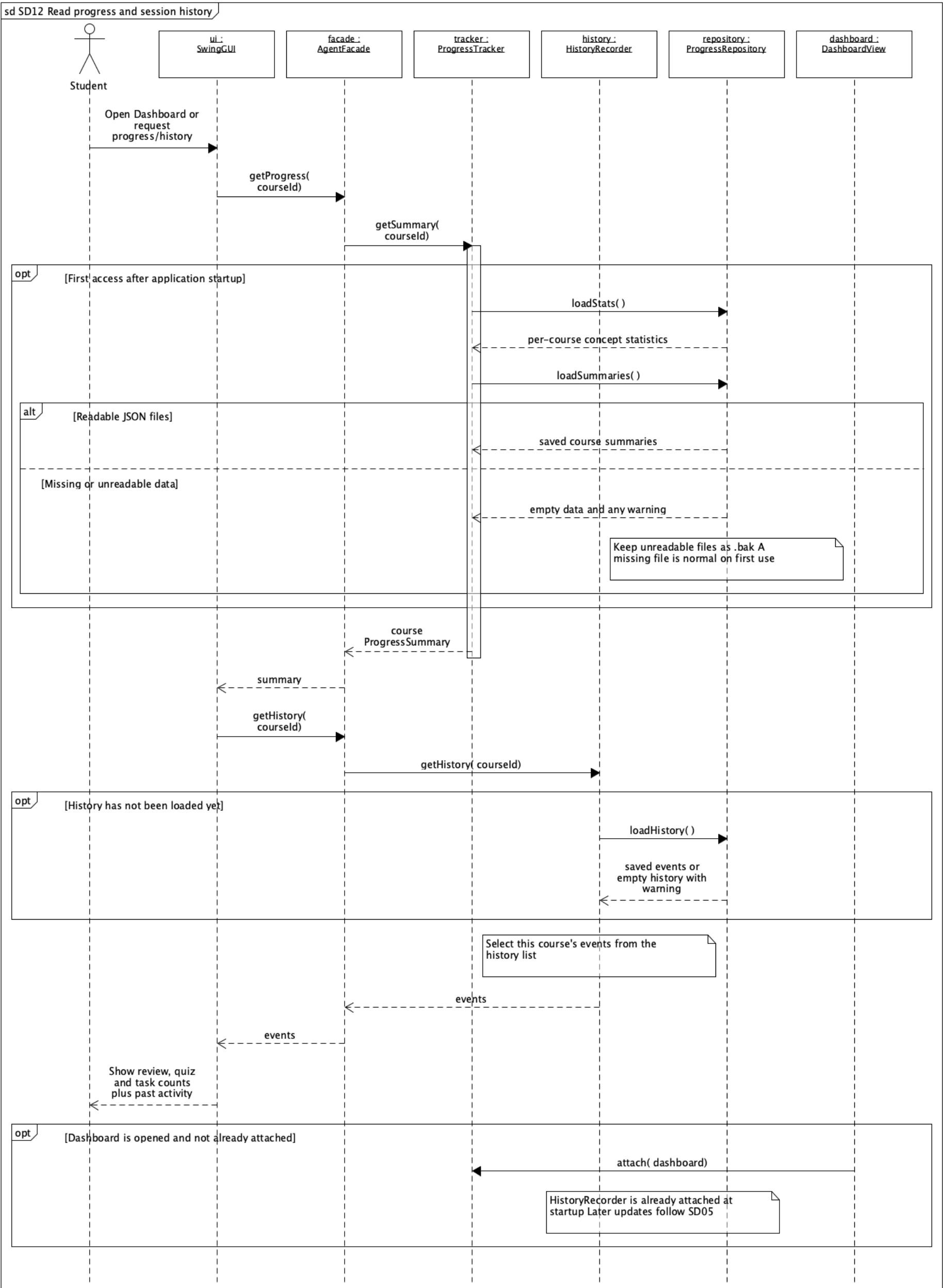
Related features: F10, F11. Related use case: UC13. Patterns: Facade, Observer.



SD11 Complete diagram. Use zoom to read detailed labels.

SD12 - Read progress and session history

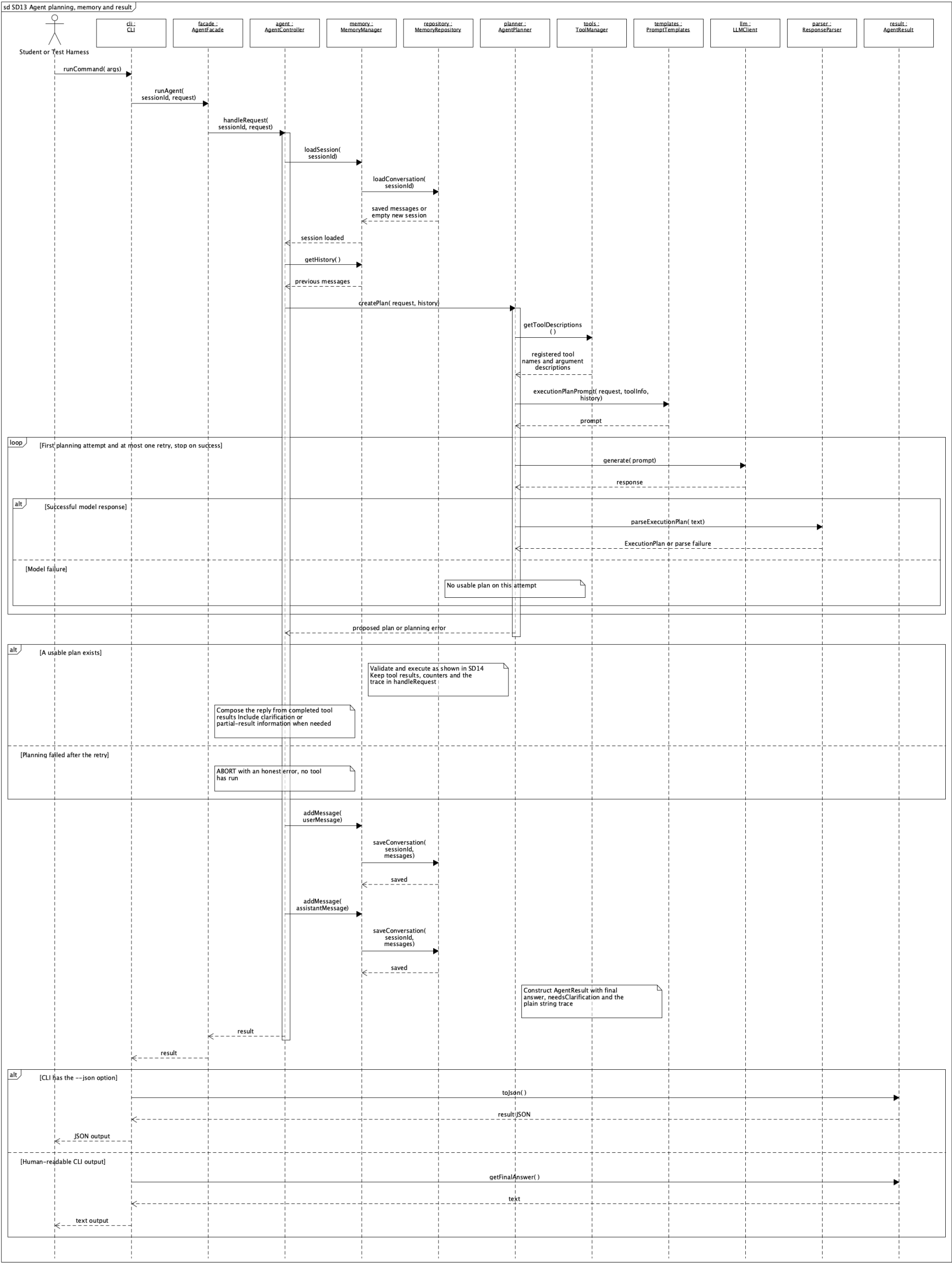
Related feature: F11. Related use case: UC14. Patterns: Facade, Observer.



SD12 Complete diagram. Use zoom to read detailed labels.

SD13 - Agent planning, memory and result

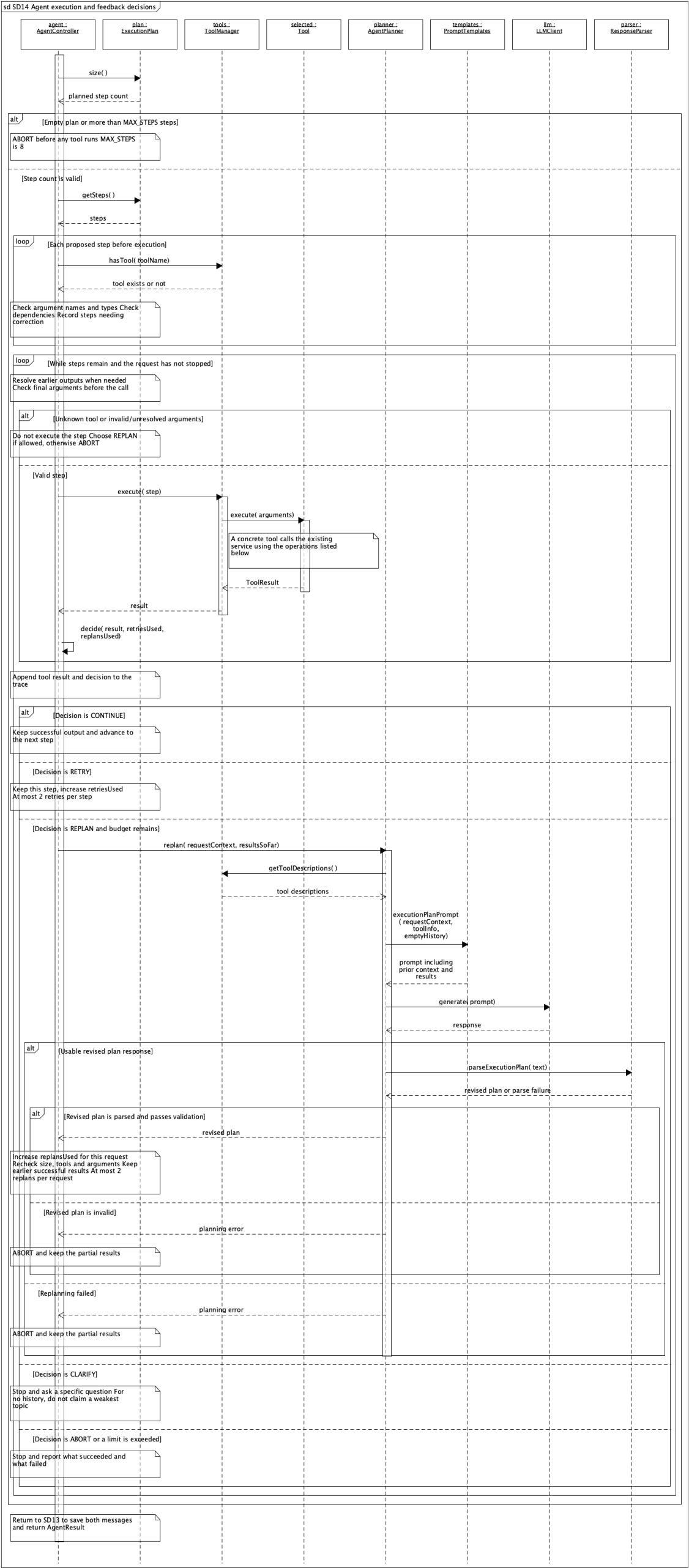
Related feature: F12. Related use case: UC15. Patterns: Facade, Adapter.



SD13 Complete diagram. Use zoom to read detailed labels.

SD14 - Agent execution and feedback decisions

Related feature: F12. Related use case: UC15. Patterns: Facade, Adapter, plus the patterns of the services selected by the tools.



SD14 Complete diagram. Use zoom to read detailed labels.

8 Feature to design traceability table

Every feature uses SwingGUI or CLI and AgentFacade. The Classes column lists the main classes beyond that common entry. OpenAIAdapter implements LLMClient; SD02 shows the external API call.

Feature	Description	AI Involvement	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Patterns
F01	Create a course; import text, Markdown or PDF with source locations.	Deterministic	UC01	CourseService Course Material Chunk CourseRepository	AgentFacade.createCourse, AgentFacade.importMaterial CourseService.createCourse, importMaterial, extractText, splitIntoChunks CourseRepository.saveCourse	SD01	Facade
F02	Produce and save a summary in the chosen style.	AI-based	UC02	SummaryService Material PromptTemplates LLMClient OpenAIAdapter OpenAIAPIClient CourseRepository	AgentFacade.summarize SummaryService.summarize PromptTemplates.summaryPrompt LLMClient.generate OpenAIAPIClient.sendRequest CourseRepository.saveCourse	SD02	Facade, Adapter
F03	Extract concepts, validate their structure and merge duplicates.	Hybrid	UC03	ConceptService Concept PromptTemplates LLMClient ResponseParser CourseRepository	AgentFacade.extractConcepts ConceptService.extractConcepts PromptTemplates.conceptPrompt LLMClient.generate ResponseParser.parseConcepts CourseRepository.saveCourse	SD03	Facade, Adapter
F04	Retrieve course chunks, answer a question and check cited chunk IDs.	Hybrid	UC04	QuestionAnswerService RetrievalStrategy KeywordRetrieval TfidfRetrieval PromptTemplates LLMClient ResponseParser CitationValidator CitedAnswer Answer Citation	AgentFacade.askQuestion QuestionAnswerService.answer RetrievalStrategy.retrieve PromptTemplates.qaPrompt LLMClient.generate ResponseParser.parseCitedAnswer CitationValidator.validate	SD04	Facade, Strategy, Adapter

Feature	Description	AI Involvement	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Patterns
F05	Generate cards; record remembered/not-yet reviews and card mastery.	Hybrid	UC05, UC06	FlashcardService Flashcard PromptTemplates LLMClient ResponseParser CourseRepository ProgressTracker DashboardView HistoryRecorder	AgentFacade.generateFlashcards, getCardsToReview, reviewCard FlashcardService.generateCards, getCardsToReview, review PromptTemplates.flashcardPrompt ResponseParser.parseFlashcards Flashcard.recordReview, getMastery ProgressTracker.recordReview, notifyObservers	SD03, SD05	Facade, Adapter (generation), Observer (review)
F06	Build an immutable quiz from validated questions of the requested types.	Hybrid	UC07; suggestion method shared with UC08	QuizService PromptTemplates LLMClient ResponseParser RawQuestion QuestionCreator, its three concrete creators Question, its three concrete question types Quiz QuizRepository	AgentFacade.createQuiz QuizService.createQuiz PromptTemplates.quizPrompt LLMClient.generate ResponseParser.parseRawQuestions QuestionCreator.buildQuestion, createQuestion QuizRepository.saveQuiz	SD06	Facade, Factory Method, Adapter
F07	Suggest a level from overall graded accuracy for a topic; allow an override.	Deterministic	UC08; used in UC07	QuizService ProgressTracker ConceptStats DifficultySuggestion Difficulty	AgentFacade.suggestDifficulty QuizService.suggestDifficulty, chooseDifficulty ProgressTracker.getAccuracy, getGradedCount	SD06	Facade

Feature	Description	AI Involvement	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Patterns
F08	Save fresh attempts, grade answers and resolve pending results without double counting.	Hybrid	UC09, UC10	QuizService Quiz QuizAttempt QuestionResult QuizGrader GradingStrategy ExactMatchGrading LLMRubricGrading PromptTemplates LLMClient QuizRepository ProgressTracker	AgentFacade.startAttempt, submitAttempt, retryPendingGrading, selfGrade the matching QuizService methods QuizAttempt.submit, getScore, getPendingCount QuizGrader.grade, regradePending GradingStrategy.grade PromptTemplates.gradingPrompt QuizRepository.saveAttempt ProgressTracker.recordQuizAttempt	SD07, SD08	Facade, Strategy, Adapter, Observer
F09	Rank topics using graded quiz results and add an AI explanation.	Hybrid	UC11	WeakTopicAnalyzer ProgressTracker ConceptStats WeakTopicReport PromptTemplates LLMClient	AgentFacade.getWeakTopics WeakTopicAnalyzer.analyze ProgressTracker.getRankedStats PromptTemplates.weakTopicPrompt LLMClient.generate	SD09	Facade, Adapter
F10	Create a valid revision schedule and record completed tasks.	Hybrid	UC12, UC13	StudyPlanGenerator CourseService ProgressTracker PromptTemplates LLMClient ResponseParser StudyPlan StudyTask StudyPlanRepository	AgentFacade.createStudyPlan, completeTask StudyPlanGenerator.generate, validatePlan, markTaskDone CourseService.getCourse ProgressTracker.getRankedStats, recordTaskDone PromptTemplates.studyPlanPrompt ResponseParser.parseStudyTasks StudyPlan.getTasksForDay, getTotalMinutes StudyTask.markDone StudyPlanRepository.savePlan	SD10, SD11	Facade, Adapter (generation), Observer (completion)

Feature	Description	AI Involvement	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Patterns
F11	Save per-course activity counts and history; refresh the dashboard.	Deterministic	UC14; updates from UC06, UC09, UC10, UC13	ProgressTracker Subject Observer ProgressSummary ConceptStats StudyEvent DashboardView HistoryRecorder ProgressRepository	AgentFacade.getProgress, getHistory ProgressTracker.recordReview, recordQuizAttempt, recordTaskDone, getSummary, getLastEvent inherited Subject.attach, detach, notifyObservers DashboardView.update, refresh HistoryRecorder.update, getHistory ProgressRepository.loadStats, loadSummaries, loadHistory, saveStats, saveSummaries, saveHistory	SD05, SD07, SD08, SD11, SD12	Facade, Observer
F12	Plan, execute tools, check results, retain conversation memory and return a trace.	AI-based	UC15	AgentController AgentPlanner ExecutionPlan PlanStep ToolManager Tool and its six implementations ToolResult MemoryManager Message MemoryRepository PromptTemplates LLMClient ResponseParser AgentResult	CLI.runCommand AgentFacade.runAgent AgentController.handleRequest, decide AgentPlanner.createPlan, replan ToolManager.getToolDescriptions, hasTool, execute Tool.execute MemoryManager.loadSession, getHistory, addMessage PromptTemplates.executionPlanPrompt ResponseParser.parseExecutionPlan AgentResult.toJson, getFinalAnswer	SD13, SD14; service interactions in SD02, SD04, SD06, SD09, SD10, SD12	Facade, Adapter; selected tools reuse the service patterns above

9 Feature implementation explanations

The explanations below describe how the proposed classes and methods realize each feature.

F01 - Course and Material Import

UC01; SD01. AgentFacade passes the request to CourseService. createCourse creates and saves a Course. importMaterial checks the file format, size and content hash, extractText reads the text with its page or line positions, and splitIntoChunks creates Chunks that keep their source location. The Material is added to the course and saved through CourseRepository. A bad file is rejected with a message and the course stays available.

F02 - Material Summarization

UC02; SD02. AgentFacade calls SummaryService.summarize. The service gets the prompt for the chosen style from PromptTemplates.summaryPrompt and sends it through LLMClient.generate; OpenAIAdapter converts the request and reply for the OpenAI API. SummaryService checks the text, stores it in Material.summary and saves the course. A timeout is retried once; an unusable reply is reported as an error.

F03 - Concept Extraction

UC03; SD03. ConceptService.extractConcepts sends the chunks in conceptPrompt and reads the reply with ResponseParser.parseConcepts. The service removes duplicates, checks the required fields and drops any parent link that forms a cycle, then saves the List<Concept> in the course through CourseRepository. Invalid output is retried with a stricter prompt; an empty result is reported to the student.

F04 - Question Answering with Verified Citations

UC04; SD04. QuestionAnswerService.answer uses a RetrievalStrategy (KeywordRetrieval or TfIdfRetrieval) to find the best chunks. If none are relevant, it says so without calling the LLM. Otherwise qaPrompt sends the chunks with their ids, and ResponseParser.parseCitedAnswer reads the answer and the cited ids. CitationValidator.validate removes ids that were not retrieved, and the remaining ids become Citations with file and page/line. This check only proves that the chunk was retrieved, not that it supports the answer.

F05 - Flashcards and Review

UC05, UC06; SD03, SD05. FlashcardService.generateCards sends flashcardPrompt, parses the reply, removes empty and duplicate cards, and saves them in the course. For review, getCardsToReview returns never-reviewed and low-mastery cards first. review calls Flashcard.recordReview, saves the course, and calls ProgressTracker.recordReview, which notifies the observers (DashboardView and HistoryRecorder).

F06 - Quiz Generation

UC07; SD06. QuizService.createQuiz sends quizPrompt with the topic, count, difficulty, question types and chunks, and ResponseParser.parseRawQuestions reads the draft questions. For each one, the matching QuestionCreator (MCQ, TrueFalse or ShortAnswer) builds and validates a Question through buildQuestion. The valid questions form a Quiz saved with QuizRepository. Invalid questions are replaced within a limit; if fewer are valid, the quiz states its actual size.

F07 - Adaptive Question Difficulty

UC07, UC08; SD06. QuizService.suggestDifficulty reads ProgressTracker.getAccuracy and getGradedCount for the topic. The private method chooseDifficulty applies simple accuracy thresholds and returns a DifficultySuggestion with the level and the reason. With no or too little history it returns MEDIUM and says so. The student can override the level.

F08 - Quiz Attempts, Grading and Explanation

UC09, UC10; SD07, SD08. QuizService.startAttempt creates a new QuizAttempt. submitAttempt stores the answers and calls QuizGrader.grade, which uses ExactMatchGrading for multiple-choice and true/false questions and LLMRubricGrading for short answers. Each QuestionResult is GRADED, or PENDING if LLM grading fails. The attempt is saved and ProgressTracker.recordQuizAttempt records each graded result once. retryPendingGrading or selfGrade resolves pending results later in the same way. Submitting the same attempt twice returns the stored result.

F09 - Weak Topic Identification

UC11; SD09. WeakTopicAnalyzer.analyze reads ProgressTracker.getRankedStats, which ranks topics by accuracy using graded quiz results only. weakTopicPrompt asks the LLM to explain the ranking.

WeakTopicReport keeps the ranking even if the explanation fails. Topics with too few results are listed as insufficient data.

F10 - Personalized Study Plan Generation

UC12, UC13; SD10, SD11. StudyPlanGenerator.generate checks the dates, gets the course through CourseService.getCourse and the weak topics from ProgressTracker.getRankedStats, and sends both in studyPlanPrompt. ResponseParser.parseStudyTasks reads the proposed tasks, and validatePlan checks dates, task durations and daily time. An invalid plan is repaired or regenerated within a limit, then saved with StudyPlanRepository. markTaskDone calls StudyTask.markDone, saves the plan and calls ProgressTracker.recordTaskDone.

F11 - Progress Tracking and Session History

UC14; SD05, SD07, SD08, SD11, SD12. ProgressTracker is the Subject. Its record methods (recordReview, recordQuizAttempt, recordTaskDone) update the per-course counts, save them through ProgressRepository, and call notifyObservers. DashboardView.update and HistoryRecorder.update read the tracker and refresh the screen or save the history. getProgress and getHistory return the counts and past events. An unreadable file is kept as .bak and the data starts empty with a warning.

F12 - Natural Language Study Assistant

UC15; SD13, SD14. AgentController.handleRequest loads the session with MemoryManager and asks AgentPlanner.createPlan for an ExecutionPlan. The controller checks the plan, then runs each step with ToolManager.execute; each Tool calls an existing service and returns a ToolResult. The private decide method chooses CONTINUE, RETRY, REPLAN, CLARIFY or ABORT with plain if/else (limits: 8 steps per plan, 2 retries per step, 2 replans per request). The final answer is made from the tool results, both messages are saved, and an AgentResult with the trace is returned.